

Final Report

Topic: DrillMain Intelligence

Smart drilling insights with next-step guidance and integrated maintenance.

Student Name: Uchechi Sebastian

Student ID: 300393092

Course: CSIS 4495 002 Applied Research Project

Section: 002

Team Lead: Uchechi Sebastian (Individual Project)

Table of Contents

Introduction	5
1.1 Problem Statement	5
1.2 Literature and Related Work	6
A. Real-time drilling data visualization.....	6
B. Wellbore schematics and trajectory visualization	6
C. Drilling optimization and predictive analytics	6
E. Non-productive time (NPT) analysis	7
1.3 Knowledge Gaps and Limitations.....	7
Gap 1 — Lack of depth-based event visualization	7
Gap 2 — Limited integration of drilling events with maintenance insights.....	7
Gap 3 — Lack of explainability in predictive systems.....	7
Gap 4 — Fragmentation across tools	7
Gap 5 — Limited accessibility and reusability.....	8
1.4 Hypotheses.....	8
1.5 Assumptions.....	8
1.6 Benefits	9
Summary of the Research Project.....	9
Changes to the Proposal	11
3.1 Change of Database Technology: PostgreSQL/MySQL → SQLite.....	11
Project Planning and Timeline.....	12
4.1 Project Timeline	12
4.2 Gantt chart – Project timeline	13
Implemented Features	14
5.1 Home screen:	14
5.2 The sign-up page.....	17
5.3 The login page.....	17
5.4 Wells Page	18

5.5 Create New Well.....	19
5.6 Drilling Dashboard.....	20
5.7 Segment Details Panel	23
5.8 Predictive Maintenance Page	27
5.9 Drilling Report Page	28
5.11 Extracted Operations Page.....	30
5.12 Extracted Operations Page.....	32
5.13 Well Location.....	33
Evaluation Technique	34
6.1 Functional evaluation.....	34
6.2 Performance evaluation	35
6.3 Analytical quality assessment.	35
Reflections/Discussions:	36
Work Date / Hours Log	38
AI Use Section	57
AI use Table	57
Conclusion	59
References:	60
Literature references.....	60
Appendix	61
Appendix A – Installation Guide	61
Appendix B – User Guide	62
Home Page	62
11.1 Home screen:	62
62	
11.2 About Page:.....	62
11.3 How it works:.....	63
11.4 Help & FAQ:.....	64
11.5 Contact us:	64
Registration	65

11.6 The sign-up page.....	65
11.7 The login page.....	65
Creation of Wells.....	66
11.8 Wells Page	66
11.9 Create New Well.....	66
Reports.....	67
11.10 uploading Reports	67
11.12 Report Details / Editing report.....	68
Dashboard.....	69
11.13 Drilling Dashboard.....	69
11.14 Segment Details Panel	70
11.15 KPI Cards.....	72
Prédictive Maintenance	73
11.16 Prédictive Maintenance Page.....	73
Summary Reports	73
11.17 Summary Reports page	73
Location	75
11.18 Well Location page	75
Appendix C – AI Prompt(s) Used.....	77

Introduction

Drilling operations in the oil and gas industry are complex, data-intensive processes that generate large volumes of operational information throughout the lifecycle of a well. During drilling, data is continuously recorded through daily drilling reports, operation logs, and event summaries, capturing information such as depth progression, operational activities, downtime events, and equipment involvement. This data plays a critical role in post-operation analysis, operational learning, and planning for future drilling campaigns.

Despite the availability of such data, interpreting drilling operations remains a challenging task. Drilling engineers often rely on textual reports, spreadsheets, and static dashboards to review what occurred during drilling. These representations can make it difficult to quickly understand how operational problems relate to specific depth intervals, how issues evolve, and how repeated events contribute to increasing operational risk. As a result, valuable insights into non-productive time, recurring operational issues, and potential maintenance concerns may not be immediately visible.

Within this context, there is a growing need for software tools that not only display drilling data but also help interpret operational patterns and provide explainable insights that support decision-making during and after drilling operations.

1.1 Problem Statement

Drilling operations produce detailed operational records that describe what occurred during the drilling process; however, transforming these records into actionable understanding remains a significant challenge. Engineers must interpret large volumes of operational data to determine where drilling problems occurred, how these problems evolved, and what operational behaviors may have contributed to increasing risk or non-productive time. When this interpretation is inefficient or unclear, valuable lessons from drilling operations may be lost.

This project is framed around the following research questions:

1. How can drilling operation data be visually organized to clearly show where operational problems occur along well depth?

2. How can recorded operational events be analyzed to identify emerging patterns that indicate increasing operational or maintenance risk?
3. How can analytics be presented in an explainable way that supports engineering judgment rather than replacing it?

These questions are important because current drilling analysis tools often emphasize data reporting rather than interpretation, making it difficult to quickly recognize patterns and understand their implications. Addressing these questions can improve post-drilling analysis, support operational learning, and contribute to more informed decision-making in future drilling operations.

1.2 Literature and Related Work

A. Real-time drilling data visualization

Commercial platforms such as Petrolink and Halliburton HalVue focus on real-time monitoring, providing dashboards for torque, weight on bit, ROP, and rig activity. These systems emphasize operational awareness and data streaming but do not provide depth-based event visualization or post-operation analysis.

B. Wellbore schematics and trajectory visualization

Tools like wellVizion offer wellbore schematics, trajectory plots, and anti-collision visualization. These systems are valuable for planning and directional drilling, but do not integrate operational events (e.g., stuck pipe, reaming, NPT) along depth.

C. Drilling optimization and predictive analytics

Platforms such as Corva and SLB DrillOps incorporate predictive analytics, real-time optimization, and automated alerts. Their focus is on improving drilling efficiency during operations using machine learning models. However, these systems are typically enterprise-scale, proprietary, and optimized for real-time rig data rather than post-run event analysis.

E. Non-productive time (NPT) analysis

NPT is widely studied as a key performance indicator in drilling operations. Research focuses on the classification of NPT events, root-cause analysis, and cost impact. However, NPT is typically analyzed in tabular or report form, not through interactive depth-based visualization.

Overall, the literature shows strong development in real-time monitoring, trajectory visualization, and predictive analytics - but these areas remain siloed and do not converge into a unified, explainable, depth-based decision-support tool.

1.3 Knowledge Gaps and Limitations

Despite the availability of advanced drilling software, several gaps remain unaddressed:

Gap 1 — Lack of depth-based event visualization

No major tool provides a simple, interactive vertical wellbore view where drilling events are mapped as **color-coded segments along depth**. Existing systems focus on real-time curves, not post-operation event visualization.

Gap 2 — Limited integration of drilling events with maintenance insights

Current predictive maintenance tools rely on sensor data or machine-learning models but do not incorporate **event history** (e.g., stuck pipe, reaming frequency, NPT clusters) as indicators of equipment stress or operational risk.

Gap 3 — Lack of explainability in predictive systems

Most commercial predictive tools operate as black boxes. Engineers often cannot see **why** a risk score increased.

The system provides **transparent, explainable logic** tied directly to recorded events.

Gap 4 — Fragmentation across tools

Drilling visualization, KPI analysis, and maintenance prediction exist in separate platforms. There is no lightweight, unified, web-based tool that combines: Drilling KPIs, depth-based event visualization, Predictive maintenance indicators, and Explainable insights.

Gap 5 — Limited accessibility and reusability

Enterprise tools are expensive, proprietary, and require real-time rig data. This system is:

- Upload-based
- Reusable across wells
- Suitable for post-run analysis
- Built with open technologies

This makes it accessible for training, engineering studies, and smaller operators.

This project addresses these gaps by developing a full-stack, reusable decision-support application that (1) visualizes drilling operations and events along a vertical wellbore (pipe) depth view, (2) computes key KPIs such as NPT and event frequency, (3) provides explainable risk indicators based strictly on recorded operational events, and (4) supports practical workflow needs such as well selection via a location map and report export.

1.4 Hypotheses

1. Depth-based visualization of drilling events will improve engineers' ability to identify operational risks compared to traditional tabular or curve-based reports.
2. Integrating drilling events and non-productive time with equipment records in a single view improves the visibility of equipment-related operational risk compared to reviewing those data sources separately.
3. Explainable insights increase user trust and adoption compared to black-box predictive models.
4. A unified, web-based platform will reduce analysis time by consolidating drilling KPIs, events, and maintenance indicators in one interface.

1.5 Assumptions

1. Drilling event data (e.g., stuck pipe, reaming, NPT) is available in structured or semi-structured form.
2. Depth information is recorded consistently across drilling operations.
3. Engineers benefit from visual, depth-oriented representations of operational events.
4. Predictive maintenance can be meaningfully informed by event frequency, severity, and clustering.
5. The system is intended for post-operation analysis, not real-time rig control.

1.6 Benefits

1. Faster identification of operational issues through intuitive depth-based visualization.
2. Improved maintenance planning by linking drilling events to equipment stress indicators.
3. Better decision-making through integrated KPIs, event history, and risk scoring.
4. Higher transparency due to explainable logic behind every insight.
5. Reusability across wells and campaigns, supporting continuous improvement.
6. A modern, full-stack engineering tool demonstrates strong technical and analytical capability.

Summary of the Research Project

This project presents the design and development of a full-stack decision-support system for drilling operations, focused on visualizing depth-based events, computing drilling performance indicators, and identifying emerging maintenance risks. The system addresses a key gap in post-operation analysis by transforming raw drilling data into an interactive vertical wellbore visualization that highlights where operational events occurred and how they affect equipment condition and overall performance.

The core objectives of the project were to:

- (1) create a depth-based visualization that maps drilling activities and events along the wellbore.
- (2) compute essential drilling KPIs such as non-productive time (NPT), washout indicators, and event frequency.
- (3) implement predictive maintenance logic that identifies increasing equipment risk;
- (4) generate explainable insights that clearly communicate why certain risks are flagged; and

(5) deliver a reusable, full-stack web application capable of supporting drilling data interpretation and reporting.

To achieve these objectives, the project followed a structured methodology. Historical drilling datasets were first prepared and standardized to represent wells, operations, events, and equipment involvement.

A backend service was developed using Python and FastAPI to handle data ingestion, storage, KPI computation, and rule-based risk analysis. The frontend was implemented in React, providing an interactive interface for well selection, depth-based visualization, KPI display, and event exploration. Analytical components including rule-based logic, statistical pattern detection, and LLM-supported explanations were integrated to ensure that all insights were transparent and traceable to underlying operational data.

Finally, the system was evaluated based on usability, clarity of visualization, and its effectiveness in supporting drilling interpretation, with the ability to generate summary reports for engineering review.

Overall, the project delivers a functional prototype that demonstrates how depth-based visualization, analytics, and explainable maintenance indicators can enhance post-operation drilling analysis and support more informed engineering decisions.

Changes to the Proposal

During the development of the project, only one modification was made to the initial proposal:

3.1 Change of Database Technology: PostgreSQL/MySQL → SQLite

- **Description of Change:**

The original proposal planned to use a full-scale relational database such as PostgreSQL or MySQL. However, during implementation, this was replaced with **SQLite** as the primary database.

- **Justification:**

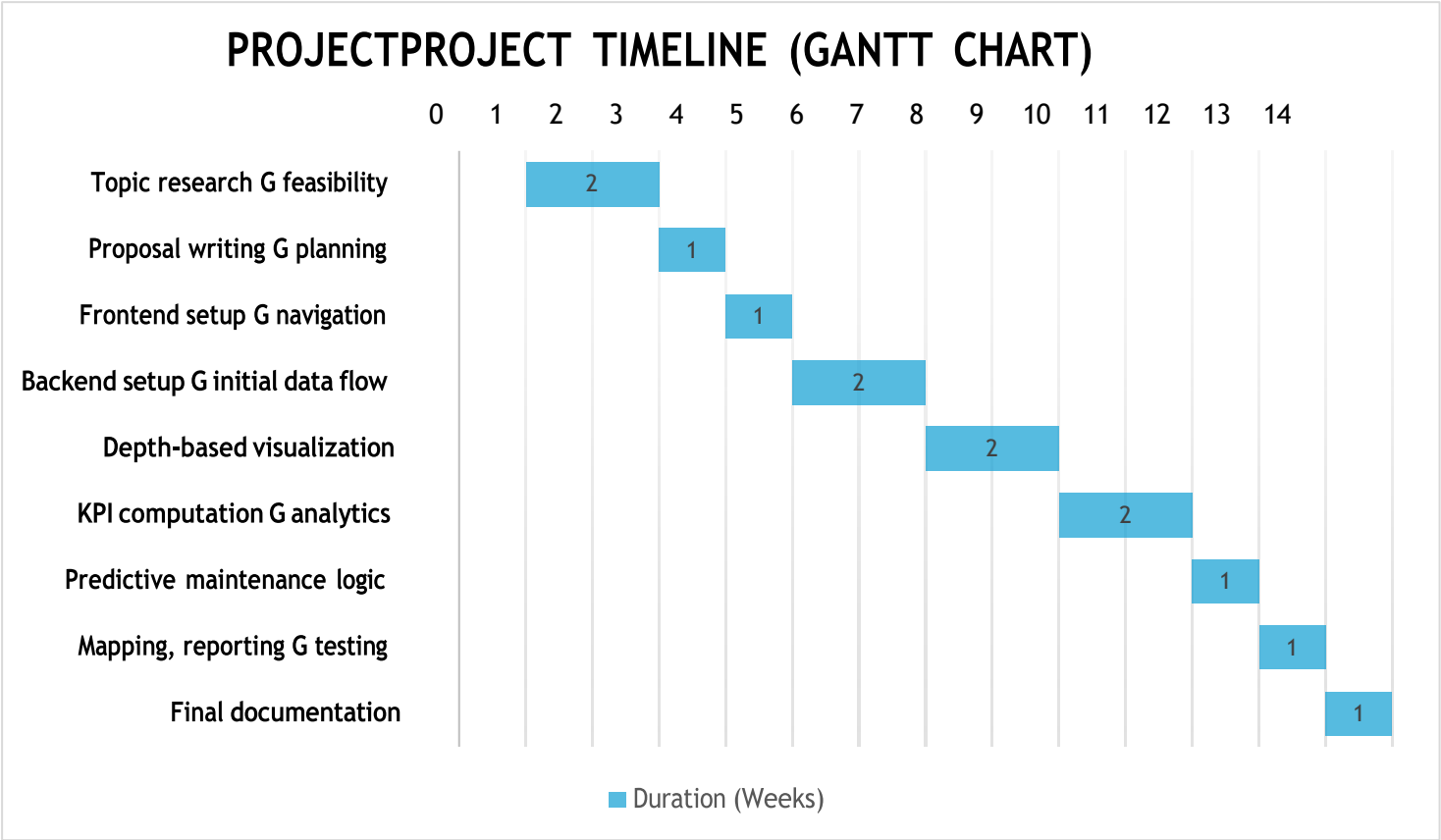
1. **Simplicity and Ease of Setup:** SQLite does not require an external database server, making it much easier and faster to configure during development.
2. **Reduced Overhead:** Removing the need for server installation, configuration, and management allowed more focus on backend logic, analytics, and visualization features.
3. **Suitable for Prototyping:** As a file-based database, SQLite is ideal for building prototypes and academic projects where rapid iteration is more important than large-scale deployment.
4. **No Change in Data Structure:** SQLite fully supports the relational schemas originally designed for PostgreSQL/MySQL, so no major redesign was required.

Project Planning and Timeline

4.1 Project Timeline

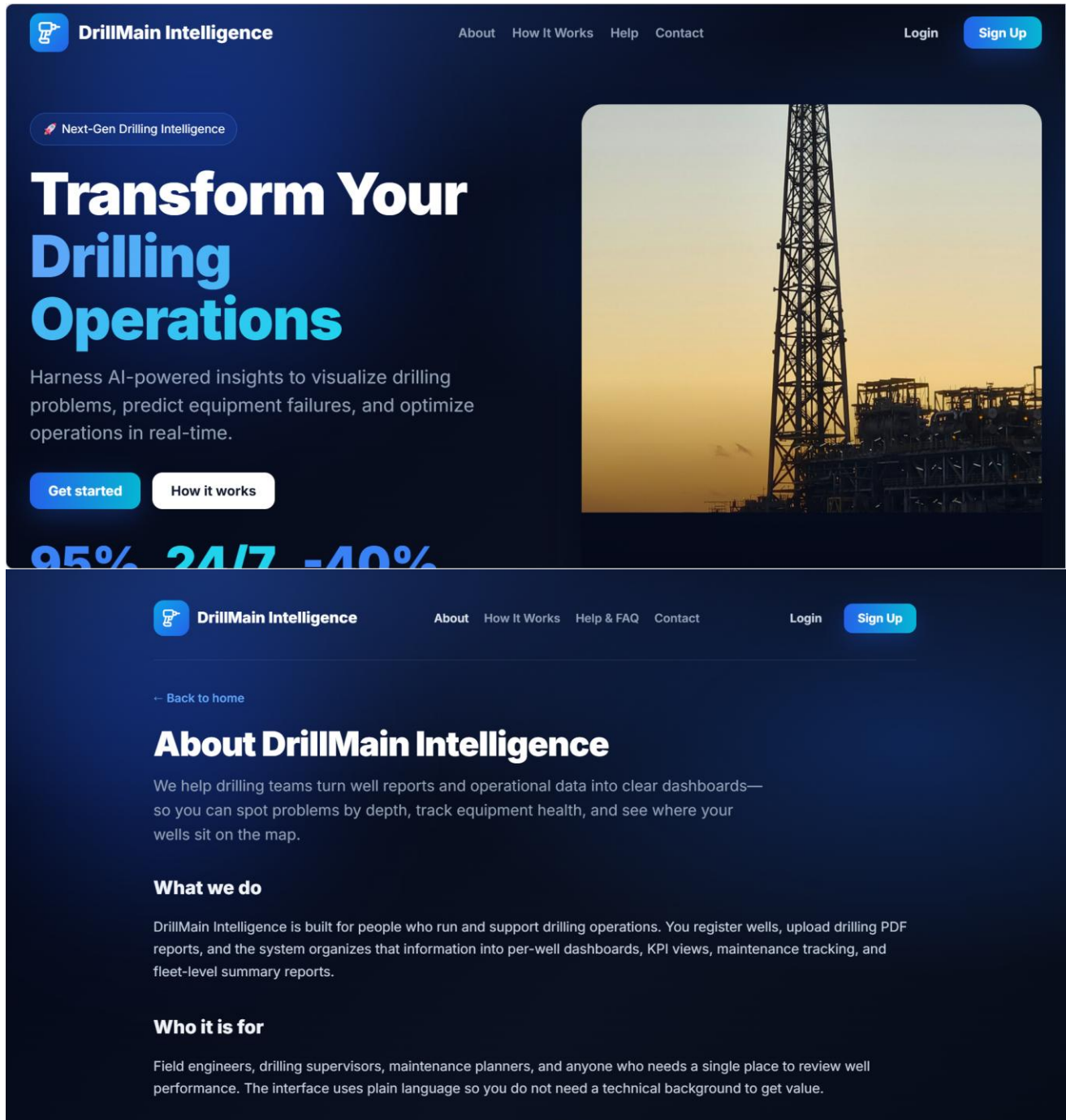
Phase	Deadline	Milestones	Deliverables
Phase 1: Topic Exploration & Domain Research	Weeks 1 - 2	Explored potential project domains Reviewed drilling operations context Identified problem area and feasibility	Project idea selection Initial domain understanding
Phase 2: Proposal Development & Planning	Week 3	Defined project scope and objectives Identified data requirements Prepared a formal project proposal	Submitted project proposal Finalized research questions and methodology
Phase 3: Frontend Development (Core UI)	Week 4	Implement the main application layout Build well selection and navigation views	Initial React frontend Well selection interface
Phase 4: Backend Setup & Initial Data Flow	Weeks 5–6	Set up backend framework (FastAPI) Designed initial database schema Implemented basic API endpoints Connected frontend to backend using sample data	Functional backend service Initial API endpoints Frontend consuming backend data
Phase 5: Depth-Based Visualization	Weeks 7–8	Implemented vertical wellbore (pipe) visualization Mapped drilling events to depth intervals	Interactive depth-based visualization
Phase 6: KPI Computation & Analytics	Weeks 9–10	Compute NPT and event-based KPIs Aggregate data by depth and operation	KPI analytics module
Phase 7: Predictive Maintenance Logic	Weeks 11	Implement rule-based risk indicators Link risk to recorded events and equipment	Predictive maintenance indicators with explanations
Phase 8: Mapping & Report Generation & Testing	Weeks 12	Implement a well location map view Enable downloadable drilling summary reports. Test system functionality.	Location-based well view Report export feature Tested application
Phase 9: Final documentation	Week 13	Prepare final documentation	Final project report

4.2 Gantt chat – Project timeline



Implemented Features

5.1 Home screen:



[← Back to home](#)

How to use the system

Follow these steps from first visit to daily use. After you sign in, the app opens on your well list.

Getting started

- 1** Create an account on the sign-up page, or **sign in** if you already have one. Wells you add are visible only under your login.
- 2** After login you land on **Wells**. Add a well with a name and location so you can find it later.
- 3** Upload a daily drilling report, or you can **input daily rig activity**.
- 4** Open a well to see its **dashboard**. You'll get KPIs and a clear view of metrics by depth/segment.

Reports and uploads

From a well, use **Reports** to browse past uploads and open report details. Use **Upload report** to add a new drilling PDF;



DrillMain Intelligence

[About](#)

[How It Works](#)

[Help & FAQ](#)

[Contact](#)

[Login](#)

[Sign Up](#)

[← Back to home](#)

Help Center & FAQ

Short answers to common questions. For a full walkthrough, read [How It Works](#).

Frequently asked questions

Do I need to install anything?

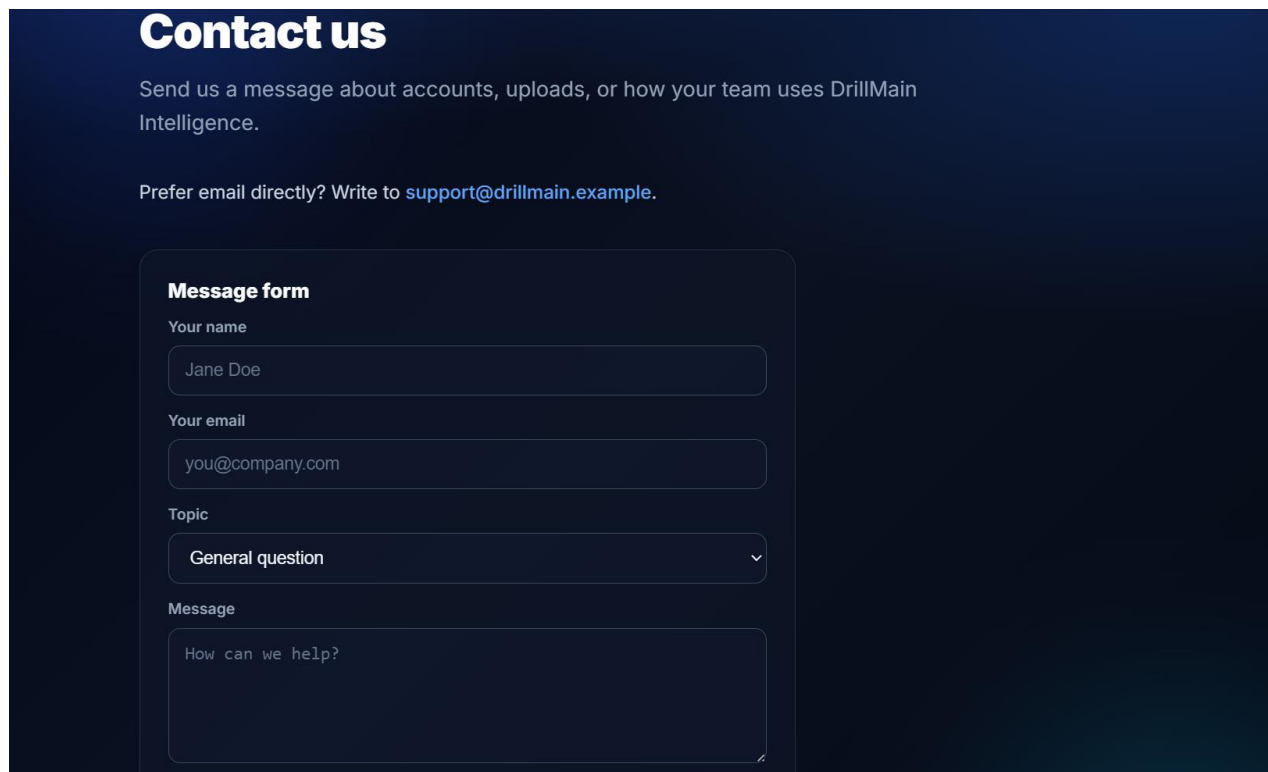
Why can't I see the Wells page?

Who can see my wells?

What file type should I upload?

What is the dashboard showing?

How do Summary Reports differ from a single-well report?



Contact us

Send us a message about accounts, uploads, or how your team uses DrillMain Intelligence.

Prefer email directly? Write to support@drillmain.example.

Message form

Your name

Jane Doe

Your email

you@company.com

Topic

General question

Message

How can we help?

The landing page introduces users to the DrillMain Intelligence platform and communicates the system’s purpose at a glance. It presents a clear value proposition—showing that the platform provides AI-powered drilling insights, event visualization, and predictive maintenance capabilities.

The page includes simple navigation (About, How It Works, Help, Contact, Login, Sign Up) and two main call-to-action buttons (“Get Started” and “How It Works”) to guide new users into the system.

A large hero section with a drilling-rig image reinforces the domain context and gives the platform a professional, industry-focused appearance. Overall, the landing page sets the tone for the application, supports user onboarding, and provides a polished entry point into the system.

5.2 The sign-up page.



Join Leading Drilling Teams

Create your free account and start using DrillMain Intelligence today.

☒ No credit card required

[← Back to Home](#)
DrillMain Intelligence

Create Account

Start optimizing your drilling operations today.

Full name

Company

Email address

Password

 [👁](#)

5.3 The login page

[← Back to Home](#)
DrillMain Intelligence

Welcome Back

Sign in to continue to your dashboard

Email Address

Password

 [👁](#)

☐ Remember me [Forgot password?](#)

Sign In

OR

Don't have an account? [Sign up for free](#)

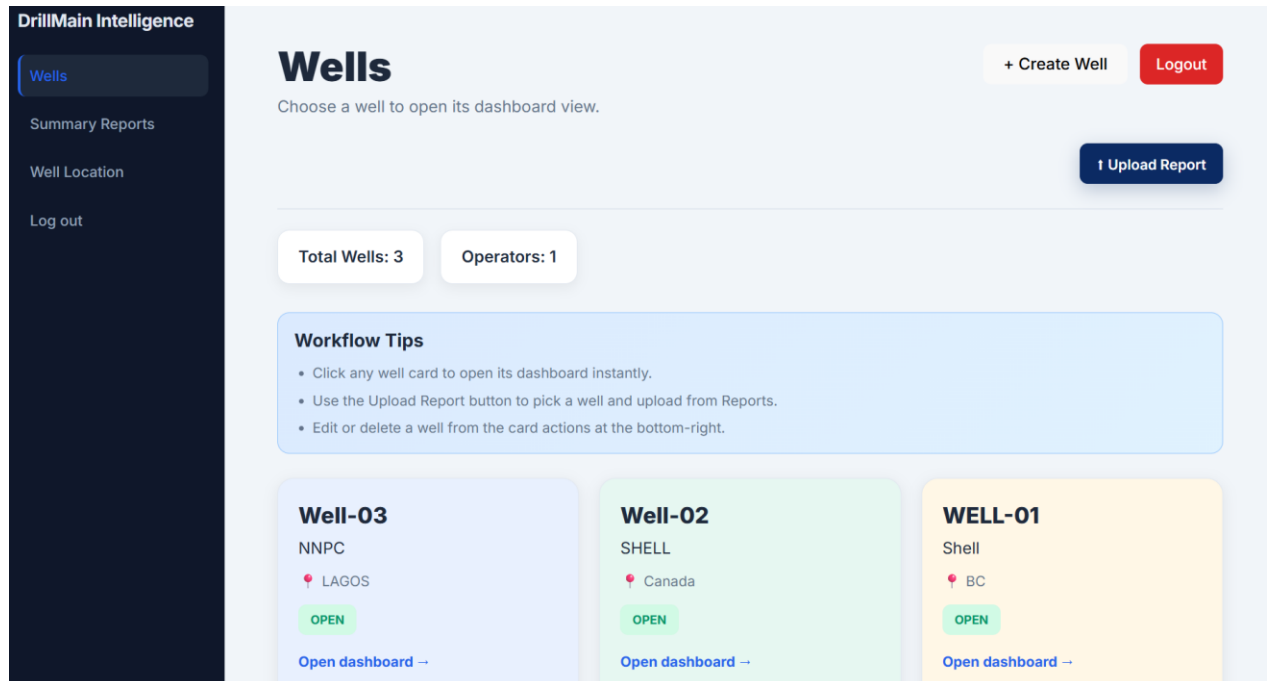


Drilling intelligence from your data

Ingest daily drilling reports, explore operations by depth, and review fleet-level KPIs—all in one place.

- Wellbore visualization
- Daily drilling PDF upload and parsing
- Per-well dashboards: NPT, risk zones, and segment metrics
- Fleet summary reports and maintenance views

5.4 Wells Page



- **Well Management Overview:** Provides a centralized place to view and manage all wells in the system. Users can quickly scan the list of available wells, their operators, and their locations before diving into detailed analytics.
- **Recently Viewed Wells:** Displays the wells the user interacted with most recently, enabling fast navigation back to active projects without searching through the full list.
- **Create New Well:** A dedicated action button that allows users to register a new well into the system. This supports onboarding new drilling projects and expanding the dataset.
- **Upload Report:** Enables users to upload daily drilling reports (PDFs). These reports are automatically parsed and converted into structured operational data for analysis.
- **Summary KPIs (Top-Right Cards):** Shows high-level statistics such as:
 - Total Wells: number of wells currently registered
 - Operators: number of unique operating companies in the system These KPIs give users a quick snapshot of system activity.

- Navigation Sidebar
- **Logout Control:** Allows users to securely exit the system, maintaining data privacy and session integrity.

5.5 Create New Well

The screenshot shows the 'Wells' application interface. A modal titled 'Create New Well' is open, allowing users to add a new well. The modal contains three input fields: 'Well ID' (with a placeholder 'e.g. WELL-03'), 'Well Name' (with a placeholder 'e.g. Bonga North'), and 'Location' (with a placeholder 'e.g. Lagos'). At the bottom right of the modal are two buttons: 'Create Well' (in blue) and 'Cancel' (in grey). In the background, the main application shows a sidebar with 'Wells', 'Summary Reports', 'Well Location', and 'Log out'. The top right has '+ Create Well' and 'Logout' buttons. Below the modal, a list of existing wells is visible, including 'LAGOS', 'Canada', and 'BC', each with an 'OPEN' button and a link to 'Open dashboard'.

A dedicated action button that allows users to register a new well into the system.

This supports onboarding new drilling projects and expanding the dataset.

```
W26_4495_S2_UchechiS > Implementation > frontend > src > pages > Wells.jsx > Wells
1  import { useEffect, useState } from "react";
2  import { useNavigate } from "react-router-dom";
3  import { apiFetch } from "../api/client";
4  import "../styles/Wells.css";
5
6  export default function Wells() {
7    const navigate = useNavigate();
8
9    // Redirect to home if not authenticated (home page appears first)
10   useEffect(() => {
11     const loggedIn = localStorage.getItem("auth");
12     if (!loggedIn) navigate("/");
13   }, [navigate]);
14
15   const [wells, setWells] = useState([]);
16   const [loading, setLoading] = useState(true);
17   const [error, setError] = useState("");
18
19   const [showModal, setShowModal] = useState(false);
20   const [newWellId, setNewWellId] = useState("");
21   const [newName, setNewName] = useState("");
22   const [newLocation, setNewLocation] = useState("");
23   const [editModal, setEditModal] = useState(false);
24   const [editWellId, setEditWellId] = useState("");
25   const [editName, setEditName] = useState("");
26   const [editLocation, setEditLocation] = useState("");
27
28 }
```


6030-6030m	Critical	NPT Event	8.0
434-434m	Critical	Flowline / Surface Lines	6.0
434-434m	Critical	Flowline / Surface Lines	5.0
402-402m	Critical	Equipment Check	4.0
6030-6029m	Critical	Well Control	3.0

Tip: Click a row to open detailed segment information.

Recommended actions

- Prioritize mitigation for 10 critical events.
- Review 1 warning event before next operation window.
- Inspect interval 6030-6030m first (highest current risk).

Flagged intervals
5

Flagged NPT
26.0 hrs

Highest risk depth
6030-6030m

Maintenance Risk

High

Prototype rule-based risk

Status

Risk

High-Risk Zones

7 segments with NPT — depth view below

0 6547 m

402-402 m
4.0 hrs NPT · Critical

434-434 m
6.0 hrs NPT · Critical

434-434 m
5.0 hrs NPT · Critical

6030-6030 m
8.0 hrs NPT · Critical

6030-6029 m
3.0 hrs NPT · Critical

6547-6547 m
1.0 hrs NPT · Critical

+1 more in reports

```

1  import { useParams, useNavigate } from "react-router-dom";
2  import { useEffect, useState } from "react";
3  import {
4    PieChart,
5    Pie,
6    Cell,
7    ResponsiveContainer,
8    BarChart,
9    Bar,
10   XAxis,
11   YAxis,
12   CartesianGrid,
13 } from "recharts";
14
15  import { apiFetch } from "../api/client";
16  import Wellbore from "../components/Wellbore";
17  import KpiCard from "../components/KpiCard";
18  import SegmentModal from "../components/SegmentModal";
19  import KpiModal from "../components/KpiModal";
20  import { getSegmentEventTypeLabel } from "../utils/segmentEventType";
21  import "../styles/Dashboards.css";
22
23  const DASHBOARD_TIMEOUT_MS = 12000;
24
25  export default function Dashboard() {
26    const { wellId } = useParams();
27    const navigate = useNavigate();
28  
```

- Depth-Based Wellbore Visualization**

Displays the entire wellbore from surface to TD, with each operation represented as a depth-accurate segment. Color coding (normal, warning, critical) helps users instantly identify operational issues such as pack-off, stuck pipe, or high drag zones.

- **Automated Event Classification**

Each segment is automatically analyzed using rule-based logic to determine severity.

This allows the system to highlight abnormal drilling behavior without requiring manual tagging.

- **Non-Productive Time (NPT) Tracking**

Aggregates NPT hours across all operations and presents the total in a dedicated KPI card. This gives engineers a quick understanding of downtime impact and operational efficiency.

- **Event Count & Severity Summary**

Shows the total number of parsed events and how many of them are classified as critical.

This helps users gauge the overall health of the drilling operation briefly.

- **High-Risk Zone Identification**

Highlights depth intervals where repeated warnings or critical events occur. This supports proactive decision-making by drawing attention to problematic sections of the well.

- **Predictive Maintenance Access**

Provides a direct link to the predictive maintenance module, where the system evaluates operational patterns to forecast potential equipment failures or drilling hazards.

- **Report Navigation**

A quick-access button allows users to jump directly to the list of uploaded daily reports for further review or auditing.

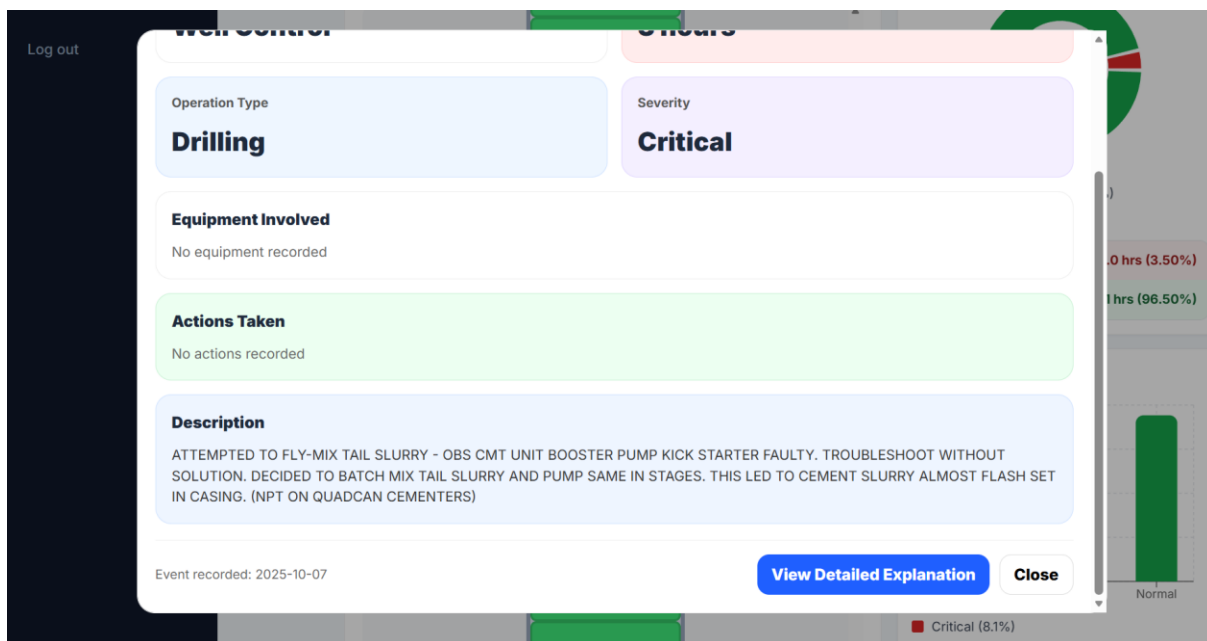
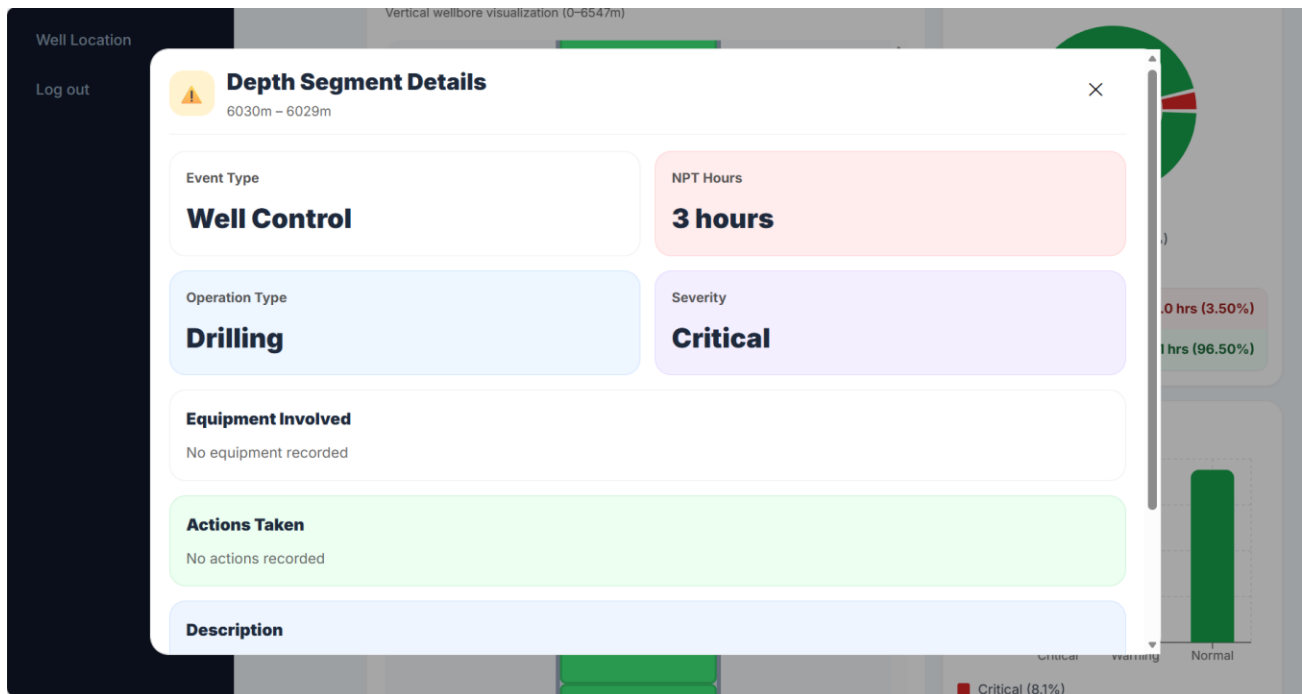
- **Live Data Integration**

The dashboard pulls structured data generated from PDF ingestion, ensuring that the visualization and KPIs always reflect the most recent drilling activity.

- **Clean, Engineer-Focused Layout**

The interface is designed to reduce cognitive load—depth markers, color-coded segments, and KPI cards make complex drilling data easy to interpret briefly.

5.7 Segment Details Panel



- **Depth-Specific Event Summary**

This panel displays detailed information for a selected depth interval in the wellbore. It allows engineers to inspect exactly what happened at a specific measured depth,

improving clarity when reviewing drilling events.

- **Operation Type Identification**

Shows the type of operation performed (e.g., Drilling, Circulation, Tripping). This helps users understand the context of the activity recorded at that depth.

- **Severity Classification**

Indicates whether the event is Normal, Warning, or Critical based on automated rule-based analysis. This classification helps users quickly assess operational risk and identify zones requiring attention.

- **NPT (Non-Productive Time) Tracking**

Displays the number of NPT hours associated with the selected event. This supports performance evaluation and helps quantify downtime at specific depths.

- **Equipment Involved**

Lists any equipment referenced in the report for that operation. If none is recorded, the system clearly states this, ensuring transparency in the extracted data.

- **Actions Taken**

Shows any corrective or operational actions documented in the report. This helps users understand how issues were addressed and whether further intervention may be needed.

- **Event Description**

Provides the raw text extracted from the PDF report for that depth interval. This preserves the original operational narrative and supports deeper interpretation of the event.

- **Additional Controls (View Explanation / Close)**

Users can open a more detailed explanation of the event or close the panel to return to the dashboard. This keeps the interface clean while still offering deeper insights when needed.

×

or

View Re

.)

.0 hrs (3.9

1 hrs (96.4%)

25

- 

- .0 h

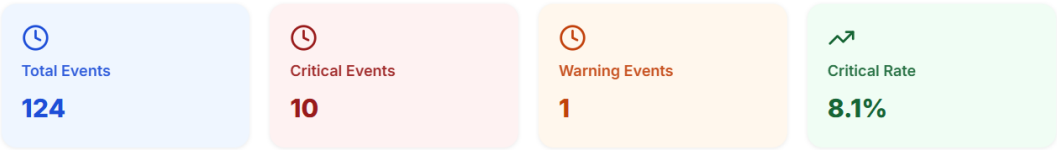
1 hrs

Close Analysis

Close Analysis

Event Analytics — NNPC

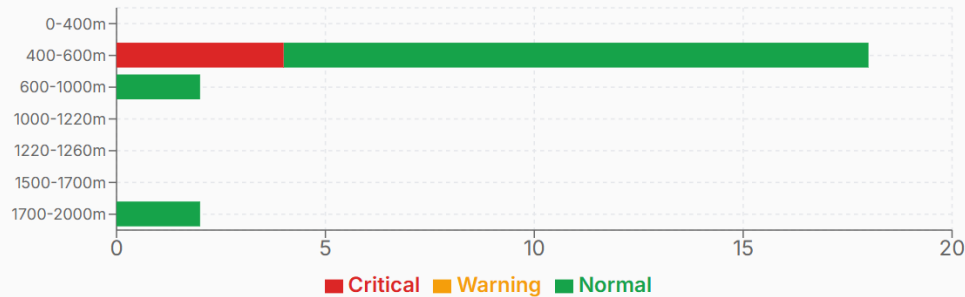
Drilling Events Analysis Across All Reports



Cumulative Events Over Time



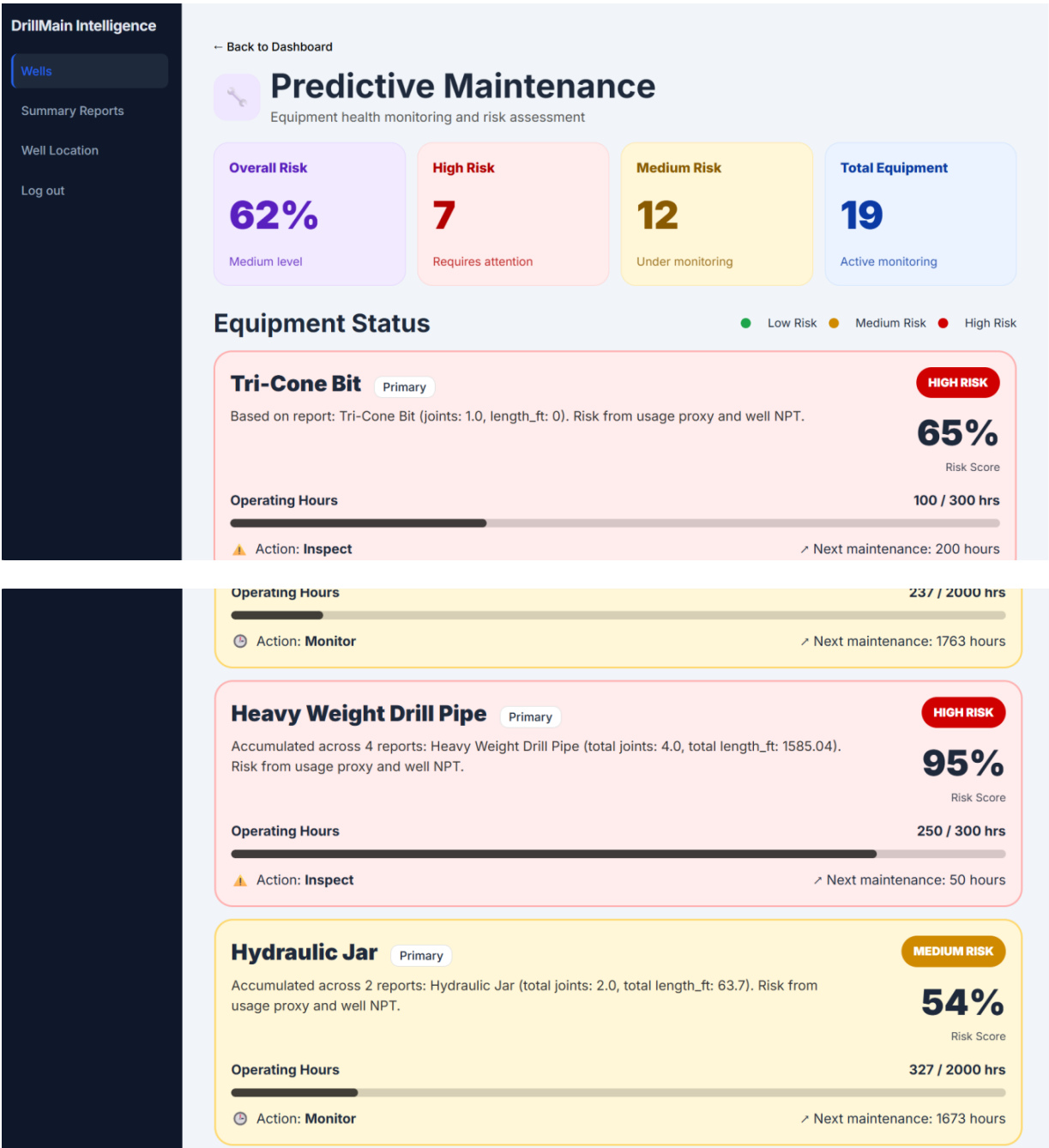
Events by Depth Range



Detailed Event Breakdown

ID	Date	Depth Range	Type	Event	Duration (hrs)	Severity
1	Sep 23	402-402m	Critical	Equipment Check	4.0	High
2	Sep 23	434-434m	Critical	Flowline / Surface Lines	6.0	High
3	Sep 24	434-434m	Critical	Flowline / Surface Lines	5.0	High

5.8 Predictive Maintenance Page



- **Overall Equipment Risk Score**
Displays a single, aggregated risk percentage representing the combined health of all monitored equipment. This gives engineers an immediate sense of operational safety and potential failure likelihood across the entire system.

- **High-Risk and Medium-Risk Equipment Counters**

Shows how many components fall into high-risk or medium-risk categories. This helps users prioritize attention, ensuring that critical equipment is addressed before it leads to downtime or safety issues.

- **Total Equipment Under Monitoring**

Indicates the number of equipment items currently tracked by the system. This provides visibility into the scope of monitoring and ensures all essential components are accounted for.

- **Color-Coded Equipment Status Legend**

Uses intuitive color markers (green, yellow, red) to represent low, medium, and high risk. This visual cue allows users to quickly interpret equipment health without reading detailed text.

- **Maintenance Recommendations**

The system provides actionable guidance such as “Monitor,” “Inspect,” or “Service,” helping engineers plan interventions proactively rather than reactively.

5.9 Drilling Report Page

DrillMain Intelligence

Wells

Summary Reports

Well Location

Log out

Reports for Well-03

Upload Drilling Report

Report Date

yyyy - mm - dd

Parser Format

NNPC Format A

Upload PDF

Click to upload or drag a PDF here

Upload Report →

- **Report Date Input**

Allows the user to specify the exact date of the drilling report being uploaded. This ensures that

each PDF is correctly tied to the operational timeline of the well, enabling accurate sequencing, filtering, and historical analysis.

- **Parser Format Selection**

Provides a dropdown for choosing the correct parsing template (e.g., *NNPC Format A*). Since different operators and rigs use different reporting structures, this option ensures the system applies the right extraction logic for each PDF, improving accuracy and reducing parsing errors.

- **Drag-and-Drop PDF Upload**

A user-friendly upload area where users can click or drag a PDF file. This is the core entry point for data ingestion — once uploaded, the system automatically extracts depth intervals, operations, NPT hours, and descriptions from the report.

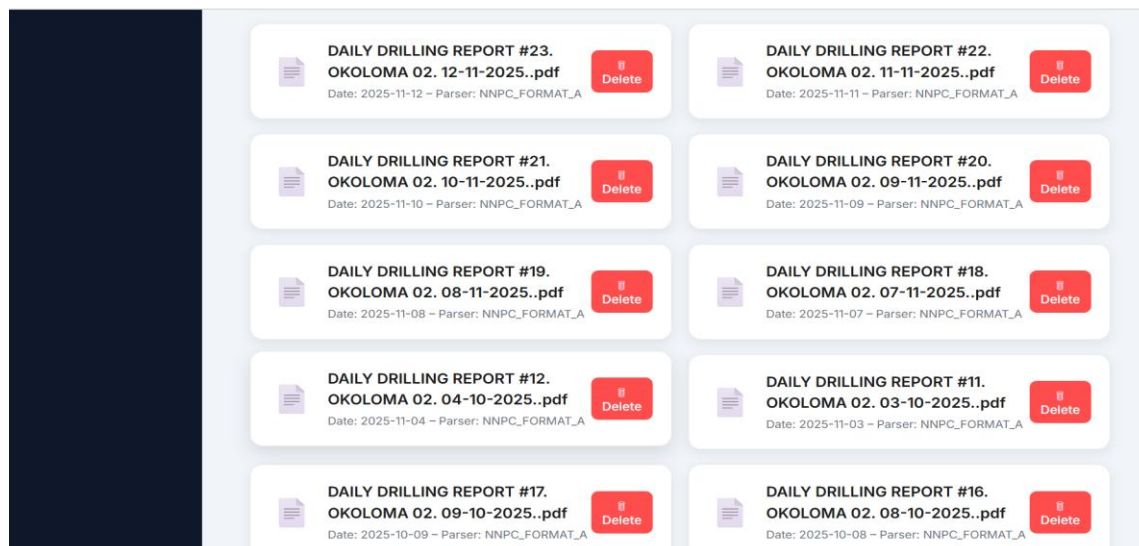
- **Upload Confirmation Button**

The **Upload Report** → button triggers the backend ingestion pipeline. It validates the file, applies the selected parser, and stores the extracted operations in the database for visualization and analysis across the dashboard.

- **Well Context Header**

The page clearly displays the current well (e.g., *Well-03*), ensuring users always know which asset they are uploading data for reducing mistakes and improving workflow clarity.

5.10 Daily Reports Page



- **Organized List of Uploaded Daily Drilling Reports**

Displays all reports associated with the selected well in chronological order. Each entry includes the report name, date, and parser format, giving users a clear overview of the dataset available for analysis.

- **Parser Format Visibility**

Shows which parsing template (e.g., *NNPC_FORMAT_A*) was used for each report. This ensures transparency in how each PDF was processed and helps users confirm that the correct extraction logic was applied.

- **Delete Report Functionality**

Each report has a dedicated Delete button, allowing users to remove outdated or incorrect uploads. This keeps the dataset clean and ensures only valid reports contribute to analytics.

5.11 Extracted Operations Page

DrillMain Intelligence

Wells

Summary Reports

Well Location

Log out

[← Back to Reports](#)

Report #20 — DAILY DRILLING REPORT #19. OKOLOMA 02. 08-11-2025..pdf

Well: Well-03
Date: 2025-11-08
Parser: NNPC_FORMAT_A

Extracted Operations

Depth From	Depth To	Operation Type	Duration	NPT	Description
6030	6030	Drilling	5.5		PREPARE THE BOP STACK IN SET BACK AREA FOR INSTALLATION. P/U AND LOWER 13-5/8" 5M x 13-5/8 " 10M DSA ON 13-3/8" 5M x 13-5/8" 5M CHH AND TIGHTENED CONNECTION BOLTS. P/U & MOVE THE 13-5/8" 10M BOP STACK FROM THE SET BACK AREA & N/U SAME ON 13-5/8" 5M x 13-5/8" 10M DSA AND TIGHTENED CONNECTION BOLTS.
6030	6030	Drilling	17.5		POWERED UP KOOMEY UNIT AND FUNCTION TESTED SAME-OK. R/U DS FRONT LADDER, CATWALK & V-DOOR CONNECTED HYDRAULIC CONTROL LINE FROM KOOMEY UNIT TO BOP. R/U BELL NIPPLE AND SECURE SAME TO THE BOP.

60306030Drilling1

(2 STD OF 5-1/2" HWDP + 13-3/8" 68 PPF CUP TESTER) AND RIH SAME BELOW THE CHH IN THE 13-3/8" CSG. CLOSE ANN & PRESSURE TESTED THE SOV & CHH CONNECTION TO 300 PSI / 5 MINS AND 2500 PSI/10 MINS- GOOD TEST POOH AND L/D CUP TESTER, 01:00 - 03:30 ==

Edit Operations

Mud details

Mud desc.	Density (ppg)	Viscosity (s/qt)	PV (cp)	YP (lbf/100ft²)	Cl⁻ (ppm)	Ca²⁺ (ppm)	pH	Pm	Pf (cc)	Mf (cc)
WATER BASED MUD	8.3	—	—	—	—	—	—	—	—	—

Edit Mud

Equipment used

No equipment data extracted from this report.

Add Equipment

- Parsed Report Metadata**

At the top of the page, the system displays key information about the uploaded report, including the report number, file name, well name, report date, and parser format. This ensures full traceability and helps users verify that the correct report was processed.
- Structured Operations Table**

The core of the page is a table that lists every operation extracted from the PDF. Each row represents a depth-based activity captured in the daily drilling report, converted into clean, structured data for analysis.
- Depth Interval Extraction**

The table includes Depth from and Depth To columns, showing the exact measured depth range for each operation. This allows engineers to correlate activities with specific sections of the wellbore.
- Operation Type Classification**

Each operation is automatically categorized (e.g., Drilling, Circulation, Tripping). This classification helps users quickly understand the nature of the activity without reading long descriptions.
- Duration and NPT Identification**

The system extracts the Duration of each operation and any associated Non-Productive Time (NPT). This supports performance evaluation and helps quantify operational

efficiency.

- **Full Description Extraction**

The Description column contains the raw operational narrative pulled directly from the PDF. This preserves all engineering details—such as WOB, RPM, GPM, torque, mud properties, and lithology—ensuring no information is lost during parsing.

5.12 Extracted Operations Page

Well Location
Log out

2025..pdf

Well: Well-03
Date: 2025-11-08
Parser: NNPC_FORMAT_A

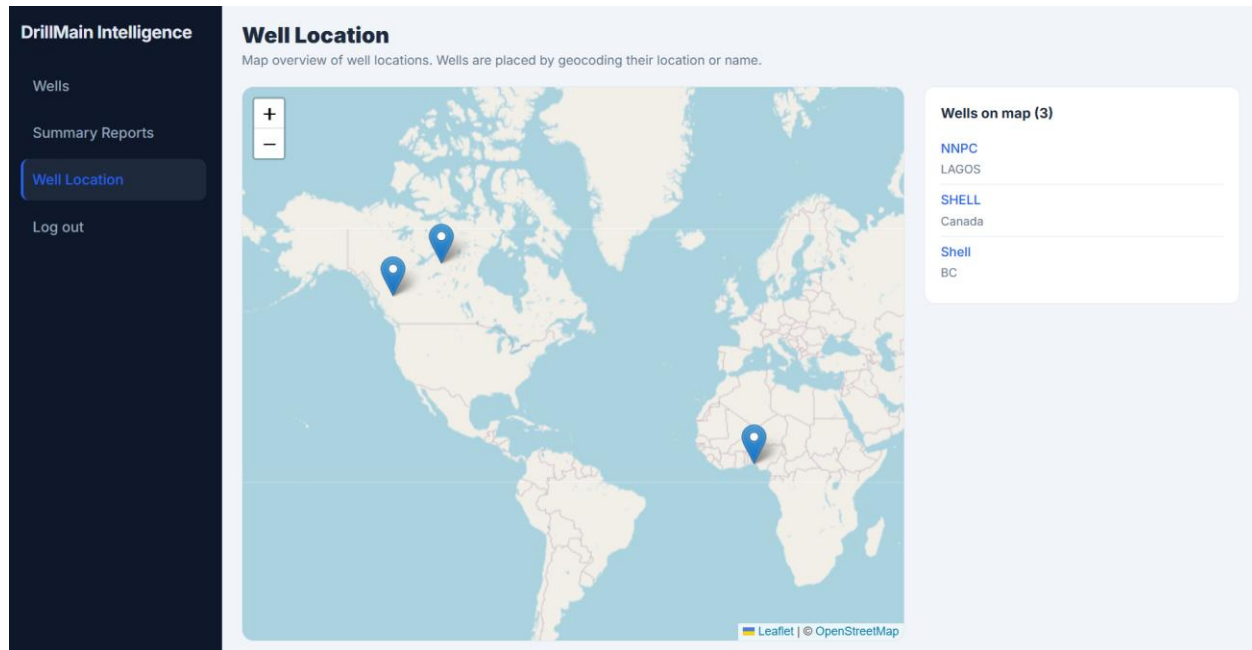
Extracted Operations

Depth From	Depth To	Operation Type	Duration	NPT	Description	Actions
6030	6030	Drilling	5.5		PREPARE THE BOP STACK IN SET BACK	Delete
6030	6030	Drilling	17.5		POWERED UP KOOMEY UNIT AND FUNCTION	Delete
6030	6030	Drilling	1		FUNCTION ALL FUNCTIONS OF THE BOP	Delete

+ Add Row Save Changes X Cancel

- **Editable Operations Table** Shows all parsed operations from the uploaded report, with fields like depth, operation type, duration, NPT, and description.
- **Inline Editing Tools** Users can modify extracted values, correct parsing errors, or refine descriptions directly in the table.
- **Add Row / Delete Row Controls** Allows adding missing operations or removing incorrect ones to keep the dataset accurate.
- **Save & Cancel Actions** “Save Changes” updates the backend with edited data, while “Cancel” discards modifications and restores the original parsed values.

5.13 Well Location



The Well Location feature provides a geographic overview of all wells stored in the system, allowing users to quickly understand the spatial distribution of drilling operations across different regions. This feature uses an interactive map interface to display each well as a geocoded marker, based on either the well's name or its recorded location.

Purpose of the Feature

The goal of this component is to give engineers and analysts a high-level spatial understanding of where operations are taking place. This supports:

- Operational planning (e.g., identifying regional clusters of activity)
- Asset management (tracking wells across different countries or basins)
- Reporting and presentations (visualizing geographic spread)
- Data validation (ensuring wells are correctly located)

How It Works

1. **Geocoding:** When a well is added to the system, its location or name is converted into geographic coordinates using a geocoding service.
2. **Map Rendering:** The frontend uses a mapping library (e.g., Leaflet) to render an interactive world map.

3. Well Markers: Each well appears as a clickable marker on the map. Selecting a marker reveals the well's name and location.
4. Well List Panel: A side panel lists all wells currently displayed on the map, making it easy to navigate or verify entries.

User Benefits

- Immediate visual context for where drilling operations are located
- Improved situational awareness for multi-region or multi-operator datasets
- Quick navigation to specific wells for further analysis
- Enhanced reporting through a clean, professional geographic visualization

Role in the Overall System

This feature complements the depth-based visualization and analytics by providing a geographical dimension to the data. While other components focus on operational performance and event analysis, the Well Location map helps users understand where those operations occur, supporting broader decision-making and reporting workflows.

Evaluation Technique

The DrillMain Intelligence system was evaluated using a combination of:

- Functional correctness
- Performance testing
- Analytical quality assessment.
- Unit testing

6.1 Functional evaluation

The functional evaluation was conducted by verifying that the system accurately processes and represents drilling data, including key performance indicators such as Non-Productive Time (NPT) and event counts. The results confirmed that the system correctly reflects operational data in the dashboard.

In addition, the system's analytical functionality was validated through the integration of a machine learning-based risk scoring model. This model computes a risk score and assigns a severity level (Low, Medium, High) for each drilling segment based on operational parameters such as NPT, depth interval, operation type, and contextual descriptions. The outputs were tested

against expected scenarios to ensure logical consistency and correctness.

Furthermore, the explanation module was evaluated to ensure that it produces meaningful and structured insights. The system uses a hybrid approach where explanations are generated using a Large Language Model (LLM) when available, with a rule-based fallback mechanism to guarantee consistent output. This ensures both high-quality insights and system reliability.

6.2 Performance evaluation

Performance evaluation focused on system responsiveness, particularly for dashboard views involving multiple wells and geolocation data, as well as real-time analytical processing.

Initial observations revealed slow loading times due to heavy API calls and sequential processing. To improve performance, optimizations such as request timeouts, reduced data loading for fleet-level dashboards, and caching of geolocation results were implemented. These changes significantly improved system responsiveness.

Additionally, the introduction of a lightweight machine learning model ensured that risk scoring and severity classification could be computed efficiently without introducing latency. For explanation generation, the system prioritizes fast rule-based analysis when necessary, while optionally leveraging LLMs for more detailed insights. This hybrid design ensures that performance remains stable even when external AI services are unavailable.

6.3 Analytical quality assessment.

The analytical quality of the system was evaluated by comparing different approaches for generating insights and explanations.

Initially, analysis relied solely on rule-based logic, which provided consistent but limited insights. This was enhanced by integrating a machine learning-based risk scoring mechanism, improving the system's ability to quantify risk and classify severity levels more effectively.

To further enhance interpretability, a Large Language Model (LLM) was incorporated to generate context-aware, human-readable explanations. The quality of these explanations was assessed based on clarity, relevance, and alignment with the computed risk levels.

A hybrid approach was ultimately adopted, where the system attempts to use LLM-generated explanations and falls back to rule-based logic when necessary. This design ensures both high-quality insights and robustness, minimizing dependency on external services while maintaining

interpretability.

Additionally, optimized data filtering and caching mechanisms were implemented to ensure that analytical computations remain efficient and scalable.

6.4 Unit testing

Unit testing was conducted on key components of the system to ensure correctness at the function level. Core functionalities such as KPI calculations, data parsing, API response handling, and machine learning-based risk scoring were tested using sample datasets.

Tests were also performed on the explanation module to verify that both LLM-based and rule-based outputs conform to the expected structure and content requirements. These tests confirmed that individual modules produced reliable and consistent outputs, supporting the overall robustness of the system.

Overall, the evaluation demonstrated that the system achieves a balance between accuracy, performance, and usability.

Reflections/Discussions:

This project provided a comprehensive end-to-end learning experience by integrating data ingestion, backend processing, frontend visualization, and user-centered design into a single workflow. One of the most important insights gained was that technical correctness alone is not sufficient; users require clear, responsive interfaces and outputs they can easily understand and trust. As a result, the dashboard and supporting pages were iteratively refined to improve clarity and usability, including redesigning KPI cards, enhancing risk visualizations, and making system outputs more context specific.

A key technical enhancement involved implementing a risk scoring pipeline for drilling segments. The system now uses a Random Forest regressor trained on a heuristic, drilling-informed risk score to produce a continuous risk score for each segment. Business rules then map this score to a severity label (Low, Medium, High), ensuring consistency and interpretability. Additionally, optional LLM-based explanations provide richer natural-language descriptions of segments when available, but the LLM is not used for scoring—it is an auxiliary layer for textual

explanations. This separation ensures that core risk assessment remains grounded in structured data while allowing flexibility for more descriptive outputs.

Another significant update addressed challenges with PDF parsing and data completeness. To ensure users could correct or refine segment information as needed, an Edit button was introduced, allowing users to adjust fields directly and ensure outputs meet their satisfaction. This reinforced the principle that automated analysis must be paired with user control to maintain trust and usability.

Balancing advanced analytics with system performance was another core challenge. Initial designs that processed all wells simultaneously or relied on sequential API calls caused slow page loads. Optimizations—including request timeouts, caching, lightweight data flows for fleet-level views, and prefetching modal explanations—significantly improved responsiveness. These improvements highlighted the importance of scalability and performance-conscious design from the start.

Iterative UX improvements also proved critical. Small refinements such as button placement adjustments, loading state enhancements, navigation clarity, and modal behavior optimization had a measurable impact on user confidence and satisfaction. Attention to reliability, including better handling of configuration and environment issues, further strengthened system robustness. Overall, the most rewarding aspect of the project was successfully integrating multiple components into a cohesive, interactive platform. The system enables users to upload drilling reports, analyze well-level and fleet-level data, edit segment information if necessary, and gain actionable insights through a clear, responsive interface. This experience reinforced the value of combining technical rigor with thoughtful design, continuous iteration, and user-centered considerations to build systems that are not only functional but also usable and trustworthy.

Work Date / Hours Log

Date	Hours	Description
17-Jan-2026	3.5	- I explored and researched possible project ideas for the course, focusing on areas related to drilling operations and data analytics. I evaluated whether the project idea was feasible to implement within the course timeline and researched whether similar work had been done before. This helped me narrow down and decide on a suitable project that aligns with my background and course requirements.
20-Jan-2026	3	- I started learning React by reviewing the basics of components, JSX, and how a React application is structured. I focused on understanding how React will be used to build the frontend of my applied project.
21-Jan-2026	3.5	- I worked on Introduction of the project: Improving the introduction, stating the problem statement clearly, outlining what questions needs to be answered. - I searched for literature and research work based on my project to find similar research on my project. I searched for gaps within this research project work.
22-Jan-2026	2.5	- I continued literature research related to drilling operations, nonproductive time, and decision support systems. I reviewed how similar systems visualize operational data and noted limitations in existing approaches, particularly depth-based visualization. - I worked on my reference and included it in my project report after the research I did.
23-Jan-2026	3	- I worked on defining the project methodology, including data sources, data collection approach, and analysis techniques. - I also clarified how simulated drilling data would be used and documented the justification for using non-proprietary datasets in the project.
25-Jan-2026	1	- I created the project timeline for the complete project for the entire semester.

26-Jan-2026	3.5	- I drew the Gantt chart for the project timeline and included it in my proposal report. - I prepared the appendix. - I went through the entire report and did a lot of corrections and restructuring to make sure the proposal fits what my project is all about.
27-Jan-2026	2	- I set up the project GitHub repository structure according to the course requirements and organized folders for implementation and documentation. - I created the frontend, backend, and data directories under the implementation folder to support a clean development workflow.
29-Jan-2026	5	- I initialized the React frontend using Vite within the repository and verified that the development server runs correctly on my local machine. This step confirmed that the frontend environment was properly configured and ready for further development. - I continued learning React by implementing the initial application layout and navigation structure. I worked on creating reusable components for the application frame (layout and navigation) and began organizing the project into pages and components to follow best practices. I separated the HTML structure (JSX) and styling (CSS) into different files to improve readability and maintainability. I also installed and configured React Router to support page navigation between the wells list, dashboard, and reports sections of the application. - I implemented the well selection page and routing logic to allow navigation between different wells and their corresponding dashboard views. I tested navigation behavior to ensure that selecting a well correctly loads the appropriate dashboard page. During this process, I encountered and resolved common React

		development issues, including file naming inconsistencies, incorrect import paths, and component casing errors. I fixed these issues by standardizing file names, correcting relative import paths, and restarting the development server as needed, ensuring the application builds and runs successfully after all fixes.
02-Feb-2026	3.5	- Built and refined Wells page with color-coded well status cards - Implemented depth-based wellbore visualization - Added clickable depth segments with event details modal - Integrated routing between wells and dashboards - Improved frontend layout and interaction flow
05-Feb-2026	4	- I set up flexible column normalization so the upload can handle different Excel header names - I added validation for wells, operations, and events during the upload process - I added row-level error reporting to make debugging easier and catch bad data - I improved the duplicate checks so existing records aren't inserted - I added logging across the upload flow for better visibility - I finished wiring up the /upload endpoint with multipart file support - I updated the utils for mapping, normalization, and safer data parsing - I prepared the backend for future analytics by making sure uploaded data is clean and consistent

06-Feb-2026	2	- I created simulated datasets and collected sample datasets from different companies so I could compare how each one structures its drilling data. - I analyzed the differences in column names, formats, units, timestamps, and category labels across these datasets.
09-Feb-2026	5	- Implemented a complete rewrite of the Excel upload pipeline for wells, operations, and events. - Fixed all column-name mismatches between the transformer output and SQLAlchemy models. - Updated the Well insertion logic to use well_name instead of the invalid name argument. - Corrected the operations insertion to use operation_type instead of the non-existent op_type. - Updated event insertion and duplicate-check logic to use the correct model field (event_time). - I rebuilt the transform_dataset() function to ensure consistent normalization, depth conversion, timestamp parsing, and category mapping. - Ensured all transformed sheets contain the required columns before insertion. - Improved error reporting for wells, operations, and events to make debugging easier. - Cleaned up duplicate imports and removed unused or inconsistent logic. - Stabilized the entire ETL flow so uploads now run end-to-end with predictable behavior. -

13-Feb-2026	5	- Created operations router - Added GET /wells/{well_id}/operations endpoint - Added GET /wells/{well_id}/segments endpoint - Converted database operations into frontend-ready wellbore segments - Implemented basic risk-level classification logic - Connected router to main.py - Implemented DailyReport model and ingestion workflow for PDF uploads - Added duplicate detection using file hash and (well_id + report_date) - Created NNPC_FORMAT_A parser using pdfplumber with table-based extraction - Extracted operation rows from Operation Summary section - Implemented depth extraction logic (MD from / MD to detection) - Linked parsed operations to DailyReport and Well - Added debugging preview fields (matched_rows_preview, debug_preview). - Backend now supports: - Creating wells - Uploading daily drilling PDF reports - Preventing duplicate uploads - Parsing operation summary rows into database - Retrieving operations via GET endpoint
14-Feb-2026	4	- Added CORS + created /wells/{well_id}/dashboard endpoint and updated React Dashboards.jsx to fetch real segments/KPIs instead of hardcoded data. - Cleaned upload router to only handle PDF ingestion and moved wells/operations/segments endpoints under /wells routes to avoid duplicate Swagger entries. - Created wells router with create and list endpoints. Cleaned upload router to handle only report uploads.

		- Added CORS + created /wells/{well_id}/dashboard endpoint and updated React Dashboards.jsx to fetch real segments/KPIs instead of hardcoded data. - Fetch wells list from backend /wells/ Clicking a well navigates to /wells/:wellId dashboard
15-Feb-2026	4.5	- Added /reports router with list and detail endpoints - Enabled fetching all uploaded reports from DB - Enabled viewing a single report with its parsed operations - Registered router in main.py - Backend now supports a full Reports Page UI - Re-enable KPI light tone styling via tone classes Convert wellbore into fixed-height scroll view Add depth cursor slider + merge contiguous normal segments for cleaner visualization. - Implemented Reports.jsx to fetch /reports from backend - Added ReportRow.jsx component for list UI - Added Reports.css styling - Enabled navigation to individual report pages - Frontend now displays all uploaded drilling reports dynamically
16-Feb-2026	4	- Implemented a dedicated UploadReport.jsx page with styled form for uploading PDF reports. - Added UploadReport route in App.jsx and integrated upload button at the well level. - Added modal-based Create Well form directly inside Wells.jsx for quick well creation. - Redesigned Report Detail page to use a professional table view instead of cards. - Added new CSS: UploadReport.css and ReportDetail.css for a clean UI experience. - I Connected all frontend pages with backend routes properly. - Prepared UI structure for full multi-well report categorization.
19-Feb-2026	6	- Updated upload_daily-report FastAPI endpoint to use Form(...)

		for well_id, report_date, and parser_type. This ensures the frontend can upload reports using multipart/form-data. - Fixed 422 validation errors and restored successful PDF ingestion. - Added DELETE /reports/{report_id} endpoint in FastAPI - Added delete button to each report card in Reports.jsx - Added confirmation dialog and optimistic UI update after deletion - Added new CSS styles for delete button and improved card layout - Ensured seamless integration with existing upload and report detail flows. - fix(reports): register DELETE endpoint correctly - Moved delete_report() outside get_report_details() - Ensures FastAPI registers DELETE /reports/{report_id} - Resolves 405 Method Not Allowed during report deletion - main(wells): add Recently Viewed wells + Quick Actions bar + UI improvements - Added Quick Actions bar (Create Well, View Reports, Upload Report) - Added stats bar and modernized layout - Added color-coded well cards, operator avatars, and status badges - Improved overall dashboard experience - fix(wells): correct quick-action routing and add well-selection modal - View Reports and Upload Report now open a modal to choose a well - Added divider line to separate quick actions from wells - Improved navigation logic and UX consistency - style(wells): redesign Create Well modal for improved UX
--	--	---

		- Added modern modal card with animations - Improved input styling and labels - Added structured footer with Create + Cancel buttons - Enhanced spacing, shadows, and visual hierarchy
20-Feb-2026	4	- implement Sign Up page and full localStorage-based authentication - Added SignUp.jsx and SignUp.css - Updated Login.jsx to validate against stored user accounts - Added /signup route and integrated with login flow - Enabled full sign-up → login → dashboard experience - added Sign Up link to Login page and complete auth flow - Added "Don't have an account? Create one" link - Linked Login → SignUp route - Updated styling for login switch link - Ensured smooth SignUp → Login → Wells navigation - Moved login/signup routes outside Layout wrapper - Added premium gradient background for auth pages - Optional glassmorphism card styling for modern UI
21-Feb-2026	2.5	- Replaced old icon with a cleaner well-bore-style SVG - Added new Home.css with modern gradient background and centered content - Updated Home.jsx to use new styling and improved structure - Ensured Home page loads without sidebar layout - Refined CTA button and feature pills for a more polished landing experience

23-Feb-2026	2	- Updated global App.css to ensure html, body, and #root all have width: 100% and height: 100% Prevented layout shrink issues caused by flex containers higher in the component tree Ensured that routed pages (e.g., Wells, Reports) now correctly expand to full available width Improved overall stability of the layout across all pages rendered inside <Layout/>
27-Feb-2026	4	- Fixed backend import error by correcting module paths in main.py(removed incorrect 'Implementation.' prefix) - Added predictive-maintenance API route with proper FastAPI router wiring - Introduced services/risk_engine.py for computing equipment risk scores - Ensured predictive.py imports use project-relative paths - Ensured predictive-maintenance results load dynamically in Maintenance.jsx - Cleaned router registration and removed duplicate predictive router include
4 -Mar-2026	6	Dashboard & KPIs - Fixed Event Count not updating by implementing proper cascade deletes. - Made all KPI cards clickable with short explanations. - Improved NPT KPI modal with a bar chart, proper labels, and correct report dates. Report Detail - Added editable mud and equipment tables under Operations, including mud descriptions. - Ensured Add/Edit buttons always show. - Resolved CORS/Vite proxy issues and added proper loading/error/editing states. Backend - Added mud_desc with automatic migration.

		- Added endpoints for updating equipment and adding mud. - Updated ingestion to store mud/equipment and return them in report details. Parser (NNPC Format A) - Extracted mud and equipment tables reliably using section-aware logic. - Prioritized tables with real data and expanded header keyword detection. Wellbore Visualization - Replaced fixed pipe height with dynamic height based on depth and segment count. - Removed CSS height constraints and prevented clipping by allowing overflow. Equipment in Segment Modal - Displayed equipment for each segment using equipmentByReport. - Passed equipment into SegmentModal and rendered a clean table when available. AI Segment Analysis - Added structured analysis output (reasons, factors, history, prevention, methodology). - Added endpoint for segment analysis and integrated it into SegmentModal with a full explanation view.
8 -Mar-2026	6	- Added optional LLM support for segment explanations with automatic fallback to rule-based logic. - Implemented prompt building and JSON-mode parsing for LLM responses. - Shortened and refined title-source text for clearer explanations. - Made both LLM and rule-based outputs more segment-specific using depth, operation, and NPT. - Removed generic contributing factors and simplified

		methodology text. - Connected the Maintenance page to the backend and removed all hardcoded data. - Added loading/error states and dynamic rendering for maintenance results. - Aggregated equipment by type on the backend so each appears only once. - Updated maintenance notes to show “Accumulated across N reports” when applicable. - Synced LLM and rule-based maintenance logic to reflect aggregated equipment data. - - Backend: Add GET /wells/summary endpoint returning per-well KPIs (depthMax, nptHours, eventCount, criticalEvents, highRiskZones, maintenanceRisk) and report_count for the Summary Reports page. - Frontend: - Added Summary Reports page at /summary-reports that lists all wells, with KPI cards, pie charts for NPT by well and maintenance risk distribution, and a "Download report (PDF)" button using the browser print dialog. - Added SummaryReports route in App.jsx and SummaryReports.css for layout and print-friendly styles. - Harden summary fetch with a 15s timeout and effect cleanup to avoid hanging when the backend is unavailable. - Fixed max() generator syntax in wells summary (parenthesize generator when using default=).
--	--	---

11 -Mar-2026	5	- Added "Well Location" nav link in Layout.jsx under Summary Reports (route /well-locations). - New WellLocations page: fetches wells from API, geocodes location/name via Nominatim, - displays OpenStreetMap (Leaflet) map with markers and popups linking to well dashboards. - Sidebar lists wells on map and wells not geocoded; map fits bounds to markers when present. - Used Leaflet only (no react-leaflet) to avoid React 19 peer dependency conflict. - Added WellLocations.css for map layout and responsive sidebar. - Added CSS variables in App.css for primary blue, sidebar, accents (success/warning/danger), and surfaces. - Added Inter font (Google Fonts); use for body and headings. - Redesigned sidebar: dark blue background (#0f172a), light text, active link with primary blue accent. - Used theme variables across Wells, Dashboards, Summary Reports, Home, Login, SignUp, Well Locations, Reports, Upload, KpiCard, and modals. - Aligned primary blue to #2563eb and use success/warning/danger accents for badges and risk states.
13-Mar-2026	3	- Improved rule-based segment analysis in ai_engine.py so normal segments and NPT events get distinct, data-driven titles that include operation type and depth range instead of all defaulting to "Segment Analysis". - Added a new POST /wells/segment-text-analysis endpoint that takes free-text description (e.g. from SegmentModal's Description section) and returns the same structured explanation shape used in the segment modal (titleSource, flaggedReason, contributingFactors, similarEventsInHistory, technicalFactors, preventionMeasures, methodology) by

		reusing analyze_segment. - Created a backend .env placeholder for OPENAI_API_KEY so LLM-based analysis can be enabled without hardcoding secrets.
15 -Mar-2026	5.5	- I extended the backend POST /wells/segment-text-analysis endpoint to accept real segment attributes (depth_from, depth_to, operation_type, npt_hours, level), ensuring the analysis engine uses accurate segment data instead of default placeholders. - On the frontend, I updated the SegmentModal so it now sends the correct segment metadata (depth range, operation type/event type, NPT hours, and level), allowing the explanation header to display the true operational context. - I aligned the wellbore visualization with backend logic by using the API-provided segment.level for color coding, ensuring the red/warning states match the AI or rule-based classification rather than relying on frontend keyword heuristics. - I improved the segment analysis output by making the “Why Was This Flagged?” and “Why It Was Flagged Critical” sections fully level-aware and removed repeated descriptions to keep the explanations concise and segment-specific. - I added _get_llm_flag_explanation(segment) so the LLM can generate a short, natural explanation of what happened and why the segment was flagged (critical or warning), and integrated it into the flaggedReason and critical-flag cards when an API key is available. - For rule-based fallback, I refined _explicit_reasons_for_level() to produce clearer flagged text and removed unnecessary quoted descriptions from Stuck Pipe and Reaming contributing factors. - I updated the main LLM prompt to require a plain-language, segment-specific flaggedReason that summarizes what happened

		and why it was flagged, ensuring the explanation is accurate, readable, and grounded in the segment's description.
16 -Mar-2026	4.3	- I added Recharts and lucide-react to the frontend to support richer analytics visualizations and consistent iconography across the dashboard. - I implemented a full NPT Analytics modal that opens when the Non-Productive Time KPI is clicked, including a structured header, a Download NPT Report button, four KPI cards, and a Recharts-powered pie chart comparing NPT vs Productive Time with a legend and summary table. - I expanded the analytics section with a bar chart for NPT by event type, line charts for daily and cumulative NPT trends, and a detailed NPT Events Breakdown table showing event type, hours, occurrences, percentage contribution, and averages. - I updated the dashboard so it now passes wellName, segments, and kpis into the KpiModal, and added full styling in KpiModal.css for the NPT header, KPI rows, pie chart legend, and events table, with a fallback to the original simple modal when NPT data is missing. - I extended the Event Count KPI to open a dedicated Event Analytics view, which mirrors the NPT layout and includes severity-based pie charts, event count by type, daily and cumulative trends, events by depth range, and a detailed event table; the Download Event Report option uses window.print for PDF export. - I refined the Detailed Event Breakdown so it now shows only critical and warning events, and updated the Event Count by Type bar chart to use severity-based colors (red, yellow, green) for clearer visual distinction.

19 -Mar-2026	5.7	- Added `frontend/src/pages/FleetWideReports.jsx` and `frontend/src/styles/FleetWideReports.css` to render the new Fleet-Wide Reports UI. - Fleet page fetches `/api/wells/summary`, then loads each active well's `/api/wells/{wellId}/dashboard` to compute fleet-wide charts (NPT comparison, cumulative trend, event type distribution, events per well, radar for top wells, and a detailed well summary table). - o Added export controls to the fleet report header: - o CSV download (table data) - o PDF export via `window.print()` (like the Summary Reports workflow) - o Full report export (JSON) of the computed fleet metrics. - Updated `frontend/src/pages/SummaryReports.jsx` to render the new fleet-wide report component (`FleetWideReports`) instead of the older single-page summary layout.
21 -Mar-2026	4.5	- Added frontend/src/utlis/segmentEventType.js with getSegmentEventTypeLabel() to classify events from whyItMatters, operation/event type, and nptHours (stuck pipe, lost circulation, well control, flowline/surface lines, directional/flow issues, reaming, casing/trip/cement, equipment check, minor delay, NPT event, then title cased operation fallback). - SegmentModal: Event Type card uses getSegmentEventTypeLabel(segment) instead of segment.eventType; Operation Type still shows raw report value. Event Count modal (KpiModal) - Event Severity Distribution: Normal slice green (#16a34a); custom outside - labels (no clipped "Normal"); label colors match slices; 3-row legend with - correct swatches; summary rows for Critical / Warning /

		Normal. - Events by Depth Range: Normal bars green; Legend formatter so labels match series colors. - Detailed Event Breakdown: pill badges for Type (Critical/Warning/Normal) and Severity (High/Medium/Low); bold duration; zebra striping on tbody. - Added utils/severityDisplay.js and styles/SeverityBadge.css for reusable - severity pills. Fleet-Wide / Summary Reports (FleetWideReports) - New "Detailed Event Breakdown (All Wells)" table (critical/warning segments) with Well ID and same badge styling as the modal. - Detailed Well Summary Status column uses the same pill styling (red/amber/green). - Table zebra rows; centered badge columns; bold duration column. Note: backend/parser/db changes (if any) are separate from the above UI work.
23 -Mar-2026	4.1	Backend - Added owner_email to wells and created a SQLite migration to update existing databases. - Implemented auth_scope with X-User-Email requirement, including get_well_for_user and assert_report_owned for ownership validation. - Added optional BOOTSTRAP_WELL_OWNER_EMAIL to assign ownership to previously unowned wells during development. - Updated wells list/summary to filter by owner; well creation now sets owner automatically. - Added ability to claim an unowned well on POST with the same well_id.

		- Enforced ownership checks across dashboards, maintenance, reports, and segment-analysis routes. - Added ownership verification before ingesting uploaded data. - Updated reports and report-details to ensure listing, filtering, updating, and deleting all respect well ownership. - Applied the same ownership checks across all well-scoped routes in the operations router. Frontend - Introduced a centralized apiFetch() that automatically sends X-User-Email from localStorage. - Added 401 handling that clears the session and redirects to login when needed. - Replaced all direct /api fetch calls across Wells, Dashboard, fleet/summary reports, maps, maintenance, reports, report detail/edit, upload, and segment text analysis with the new authenticated fetch layer. - Updated logout flow to clear recentWells to prevent stale data after session changes.
27 -Mar-2026	4.5	- Home / How It Works / Login - Removed hero overlays from home image - Added “How to use the system” in the home page - Added contact us in the home page - Redesigned the signup page - Full login page redesign to split-screen style

30 -Mar-2026	4.3	Dashboard UX - Risk Zones KPI redesigned to show depth intervals (not just count) - Replaced the KPI Cards on the NPT and Event counts to donut and pie charts instead of normal counts for both. - Added small “See more” cues on clickable KPI cards - Repositioned Maintenance Risk card to reduce empty layout space. Wells page - Per-user recent logic fixed - Added edit/delete well actions (+ confirm before delete) - Removed separate Upload page route and unified flow via Report - Upload button/modal styling improvements - Replaced “Recently Viewed” with workflow tips
1 -April-2026	5	Validation/tests run - Repeated frontend builds (npm run build) after major edits: test passed - Backend syntax checks (python m py_compile on changed backend modules): test passed - Lint checks on edited frontend files (ReadLints): no new lints reported. Segment analysis quality - Technical factors/prevention now derived from actual depth description (less generic repetition) - Torque/drag-related text now flags at least warning Performance fixes - Fleet reports no longer blocked by one slow well

		- Well locations now render faster with geocode cache + background geocoding - Well location map enlarged to fill page with side list retained
7 -April-2026	5	- Removed sidebar link from frontend/src/components/Layout.jsx - Removed app route and import from frontend/src/App.jsx - Deleted AI Assistant frontend files: - frontend/src/pages/AiAssistant.jsx - frontend/src/styles/AiAssistant.css - Removed backend chat router registration/import from backend/app/main.py - Deleted backend chat API file: - backend/app/routers/chat.py - Updated wording in backend/.env.example to remove AI Assistant mention - Fixed slow loading pages for dashboard

AI Use Section

AI use Table

AI Tool	Version / Account	Specific Use	Value Added
ChatGPT	GPT-5.2	Assisted with refining the project idea, structuring the proposal, improving academic writing, and clarifying the research design, methodology, and timeline.	Helped organize thoughts clearly, improve academic clarity, and ensure the project scope and structure align with the course.
Copilot	Free	Summary of Relevant Literature and Existing Research	Helped to clearly identify the existing research on the project and showed how the project differs from various working projects.
ChatGPT	GPT-5.2	Used as learning support while developing the React frontend, including understanding React components, JSX structure, routing with React Router, file organization, and troubleshooting common setup and import issues during implementation.	Supported faster understanding of React concepts, helped resolve configuration and import errors, and improved confidence in implementing the frontend correctly within the project timeline.
Figma AI	Free	Used to generate UI layout suggestions and design inspiration for the drilling dashboard, including card layout, spacing, and visual hierarchy.	Helped to clearly identify the existing research on the project and showed how the project differs from various working projects.
Copilot (Backend Development)	Free	Assisted with setting up the FastAPI backend, configuring SQLite, creating database models, organizing folder structure, and planning the data-upload workflow.	Accelerated backend setup, ensured clean architecture, and improved clarity on how data flows from upload → database → analytics → frontend.

Copilot (Backend Development)	Free	I worked through the Excel upload pipeline, including column normalization, validation, duplicate handling, row-level error reporting, and debugging the upload endpoint.	It helped me break down the errors clearly, understand what was causing the failures in the upload pipeline, and refine the normalization and validation flow so the backend is more stable, predictable, and better prepared for the analytics features I'll build next.
Cursor	Chat	Implemented the Summary Reports feature: new backend endpoint GET /wells/summary (per-well KPIs and report count), new frontend page at /summary-reports with well cards, pie charts (NPT by well, maintenance risk distribution), and "Download report (PDF)" via print; added route and styles; fixed loading hang with a fetch timeout and backend syntax error (max() generator).	It helped me deliver full-stack feature (API, React page, styling, print layout) and fixed runtime/syntax issues so the Summary Reports page loads and the backend start correctly.
Copilot	Free	Helped me design and implement the new Well Locations feature (Leaflet map, geocoding, responsive sidebar), refine the segment-analysis pipeline with real metadata + LLM/rule-based improvements, and build full NPT/Event analytics using Recharts, KPI cards, charts, and PDF export.	Enabled me to deliver multiple full-stack features faster, improve UI consistency, enhance analysis accuracy, and ship advanced analytics views that make operational insights clearer and easier to explore.
Copilot	Free	Helped me build the Fleet-Wide Reports system, cross-well analytics (NPT trends, event distributions, radar charts), improved segment classification, upgraded severity/UI components, and implement full well-ownership enforcement across backend routes	Enabled me to deliver complex reporting features faster, improve data accuracy and access control, streamline UI consistency, and ship a more polished analytics experience across the entire platform.

Conclusion

This project successfully delivered a Drilling Operations Decision Support System that transforms unstructured daily drilling reports into actionable intelligence. Through automated PDF parsing, structured data storage, depth-based wellbore visualization, KPI dashboards, and a predictive maintenance module, the system provides engineers with a clear, data-driven view of well performance.

The development process integrated both backend and frontend engineering, including authentication flows, report management, interactive dashboards, and editable operation tables. By combining clean architecture with intuitive UI design, the platform demonstrates how modern data engineering and analytics can significantly enhance drilling oversight, reduce manual workload, and support proactive decision-making.

Overall, the project showcases a scalable foundation for future enhancements such as advanced machine learning models, multi-well analytics, and real-time operational monitoring.

References:

Literature references

WellVizion - Well schematics diagramming solution. (n.d.). <https://wellvizion.com/>

Admin. (n.d.). *PetrolInk: Data Visualization in Drilling Operations – Petrolink.*

<https://www.petrolink.com/data-visualization-in-drilling/>

HalVue™ real-time viewer. (n.d.). Halliburton. <https://www.halliburton.com/en/products/halvue-real-time-viewer>

Barakat, M., Ela, M. a. E., & Khalaf, F. (2021). Integrating risk management concepts into the drilling non-productive time. *Journal of Petroleum Exploration and Production Technology*, 11(2), 887–900. <https://doi.org/10.1007/s13202-020-01059-0>

Liu, S., Feng, Y., Wang, X., & Yan, P. (2021). Cross-Platform Drilling 3D Visualization system based on WebGL. *Mathematical Problems in Engineering*, 2021, 1–18.

<https://doi.org/10.1155/2021/5516278>

Corva.Ai. (n.d.). *AI Predictive Drilling Automation Software | Corva.* Corva.

<https://www.corva.ai/energy/predictive-drilling>

Appendix

Appendix A – Installation Guide

Installation Instructions

This section explains how to install and run the project on your machine, including both backend (Python) and frontend (React).

Backend (Python)

1. Navigate to the Backend Folder: `cd Implementation/backend`
2. Install Python: Ensure you have Python installed:
 - Check if you have installed python using: `py --version`
 - If you have not installed python, download Python from the official website: <https://www.python.org/downloads/>
 - During installation, check:
 - Add Python to PATH
 - Ensure pip and venv are selected
3. Create and Activate a Virtual Environment:
 - Create: `py -m venv venv`
 - Activate (Windows): `venv\Scripts\activate`
 - If PowerShell blocks npm scripts, allow them for this session(run this below): `Set-ExecutionPolicy -Scope Process -ExecutionPolicy Bypass`
 - Activate (macOS/Linux): `source venv/bin/activate`
4. Install Dependencies:
 - `pip install -r requirements.txt`
5. Run the Backend Server: `uvicorn app.main:app --reload`

Frontend (React)

1. Install Node.js: Verify installation:
 - `node -v`
 - `npm -v`
 - If you do not have Node.js, Download from: <https://nodejs.org>
 - Make sure Add to PATH is checked during installation.
2. Navigate to the Frontend Folder: `cd frontend`
3. Install Frontend Dependencies:
 - `npm install`
 - If PowerShell blocks npm scripts, allow them for this session(run this below): `Set-`

ExecutionPolicy -Scope Process -ExecutionPolicy Bypass

4. Start the React Development Server: `npm run dev`

The frontend will be available at: <http://localhost:5173/>

5.  Running the Full Demo Once both servers are running:

Backend runs on something like: <http://localhost:8000>

Frontend runs on: <http://localhost:5173/>

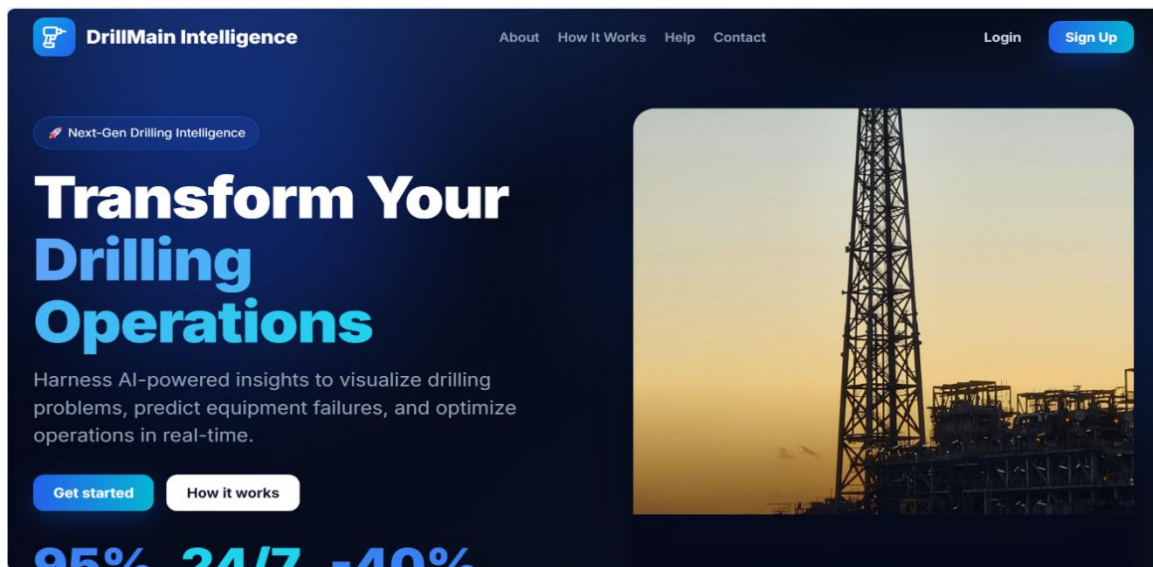
The frontend will automatically communicate with the backend if your API URLs are configured correctly.

Appendix B – User Guide

Home Page

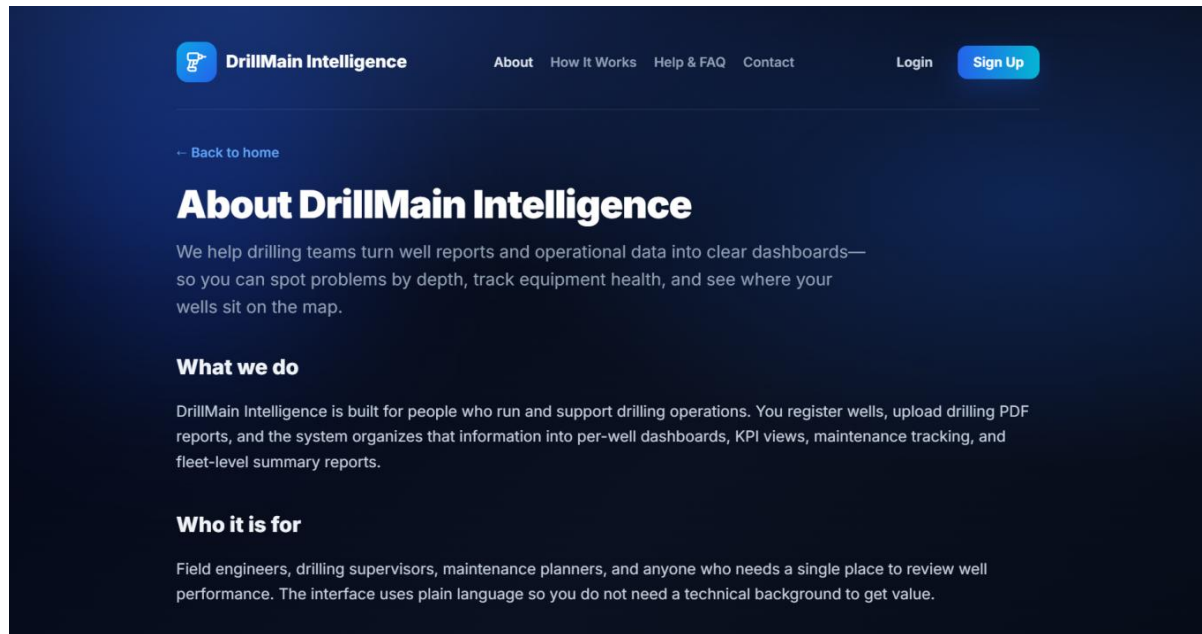
11.1 Home screen:

- The first thing you see on your screen after loading the website is the home page (see pictures below).



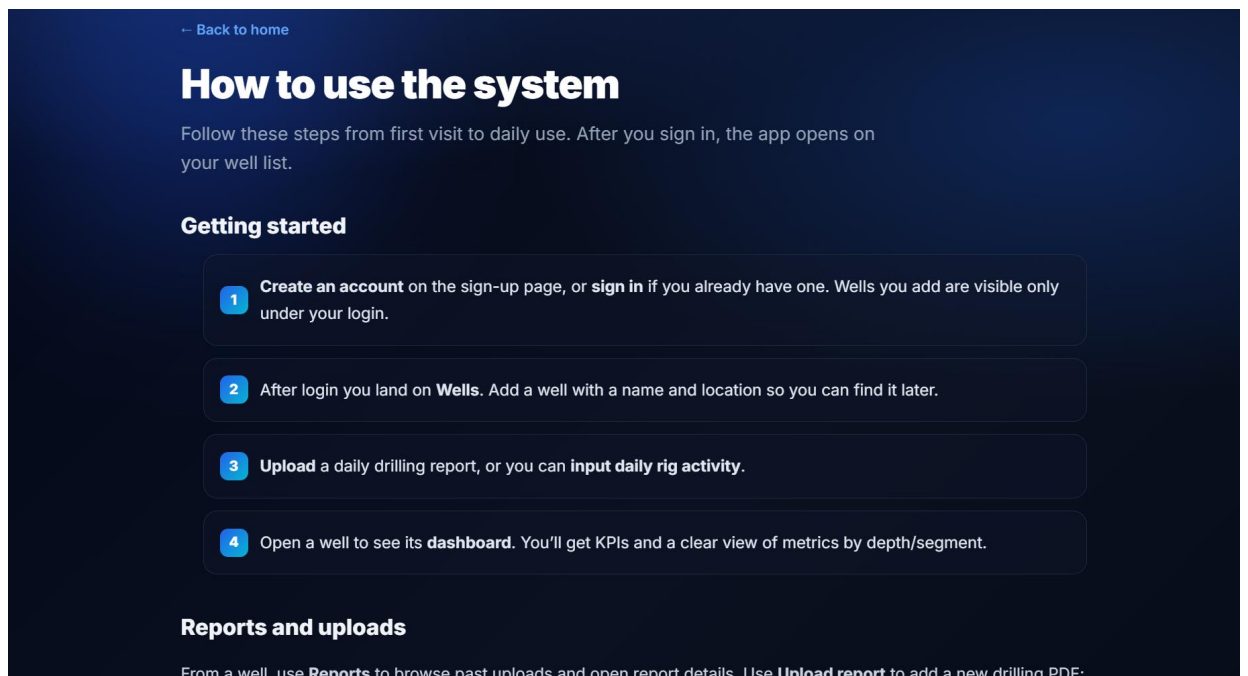
11.2 About Page:

- On the Home page, click on “About” link at the top of the page. This page explains what DrillMain Intelligence gives you a brief introduction of what the app is all about.



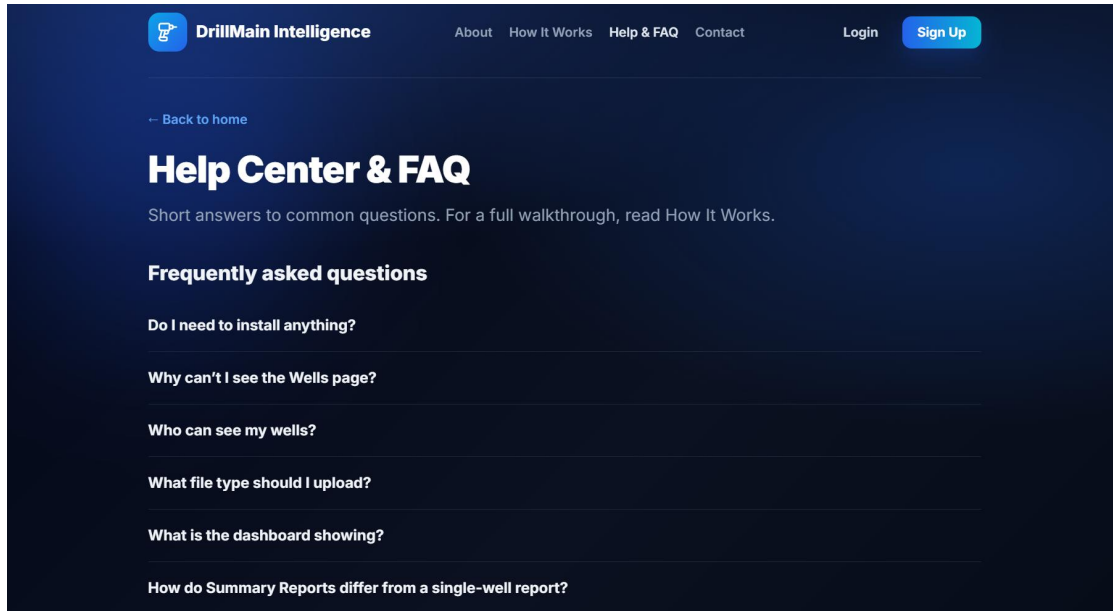
11.3 How it works:

- On the Home page, click on “How it works” link at the top of the page. This page gives a brief overview on how to use the system.



11.4 Help & FAQ:

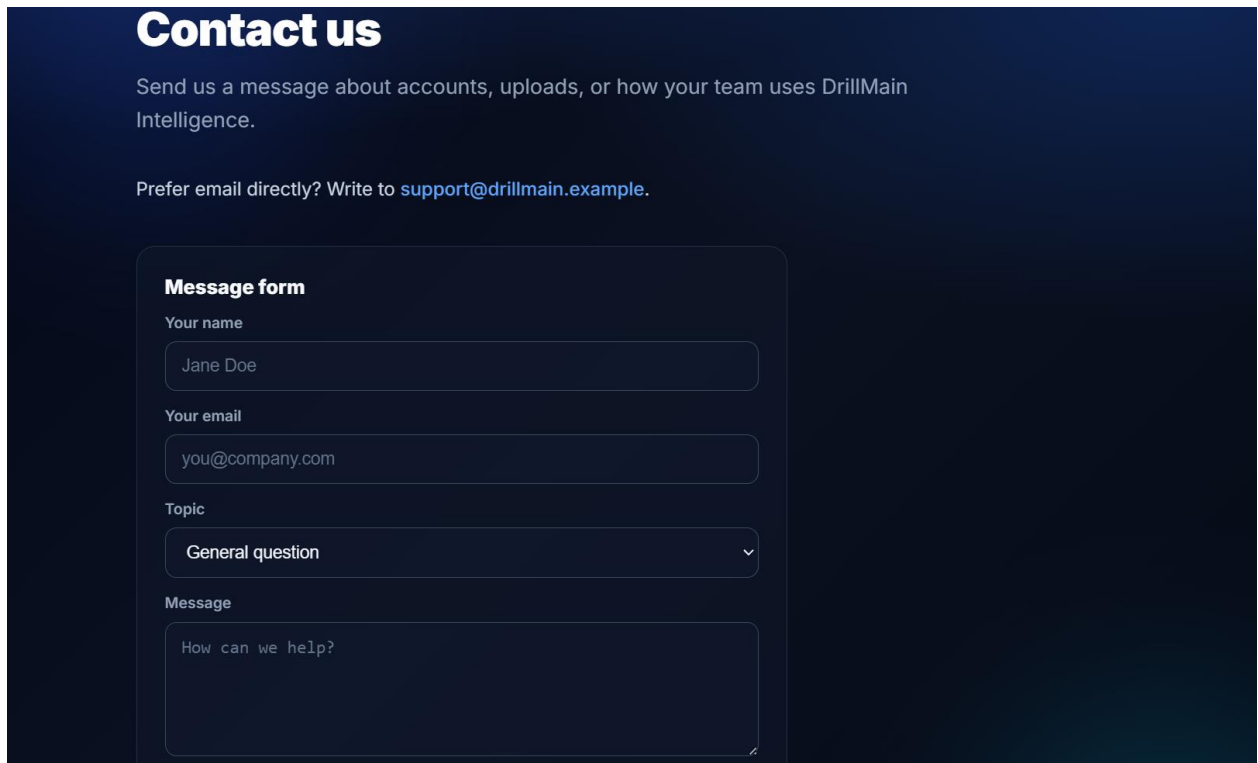
- On the Home page, click on “About” link at the top of the page. This page explains what DrillMain Intelligence gives you a brief introduction of what the app is all about.



The screenshot shows the 'Help Center & FAQ' page of DrillMain Intelligence. The page has a dark blue header with the logo and navigation links: About, How It Works, Help & FAQ, Contact, Login, and a Sign Up button. Below the header, there is a link to 'Back to home'. The main heading is 'Help Center & FAQ', followed by a subtitle: 'Short answers to common questions. For a full walkthrough, read How It Works.' Below this is a section titled 'Frequently asked questions' with a list of questions: 'Do I need to install anything?', 'Why can't I see the Wells page?', 'Who can see my wells?', 'What file type should I upload?', 'What is the dashboard showing?', and 'How do Summary Reports differ from a single-well report?'.

11.5 Contact us:

- On the Home page, click on “About” link at the top of the page. This page explains what DrillMain Intelligence gives you a brief introduction of what the app is all about.

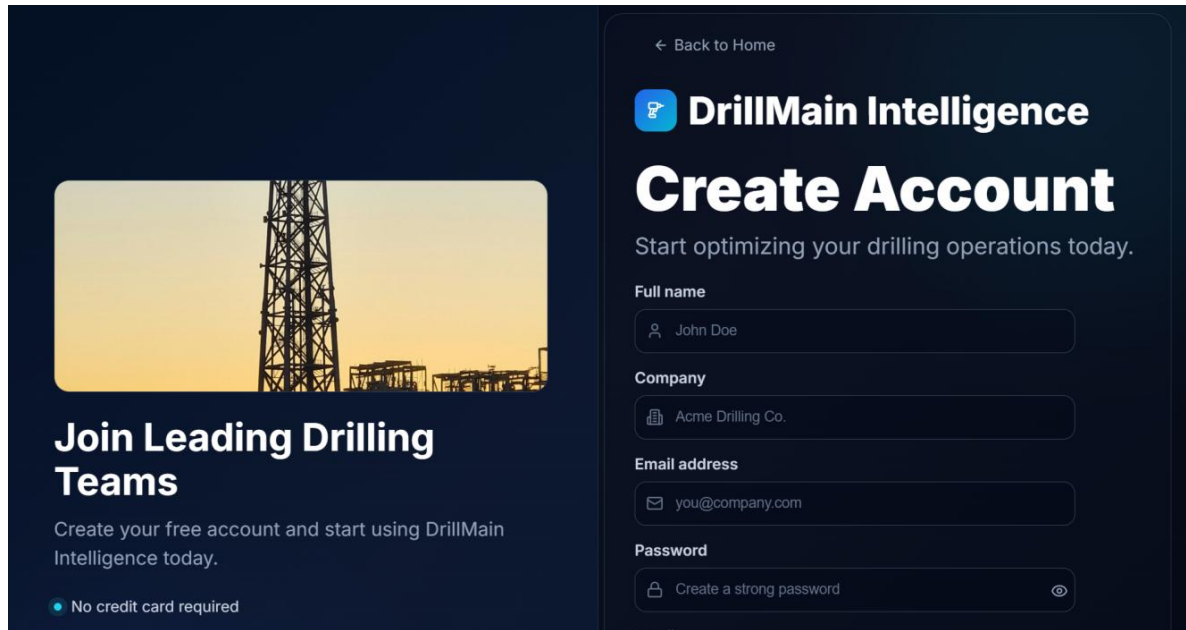


The screenshot shows the 'Contact us' page of DrillMain Intelligence. The page has a dark blue header with the title 'Contact us'. Below the title is a subtitle: 'Send us a message about accounts, uploads, or how your team uses DrillMain Intelligence.' Below this is a link to 'support@drillmain.example'. Below the link is a section titled 'Message form' with a list of input fields: 'Your name' (with the value 'Jane Doe'), 'Your email' (with the value 'you@company.com'), 'Topic' (with a dropdown menu showing 'General question'), and 'Message' (with the value 'How can we help?').

Registration

11.6 The sign-up page.

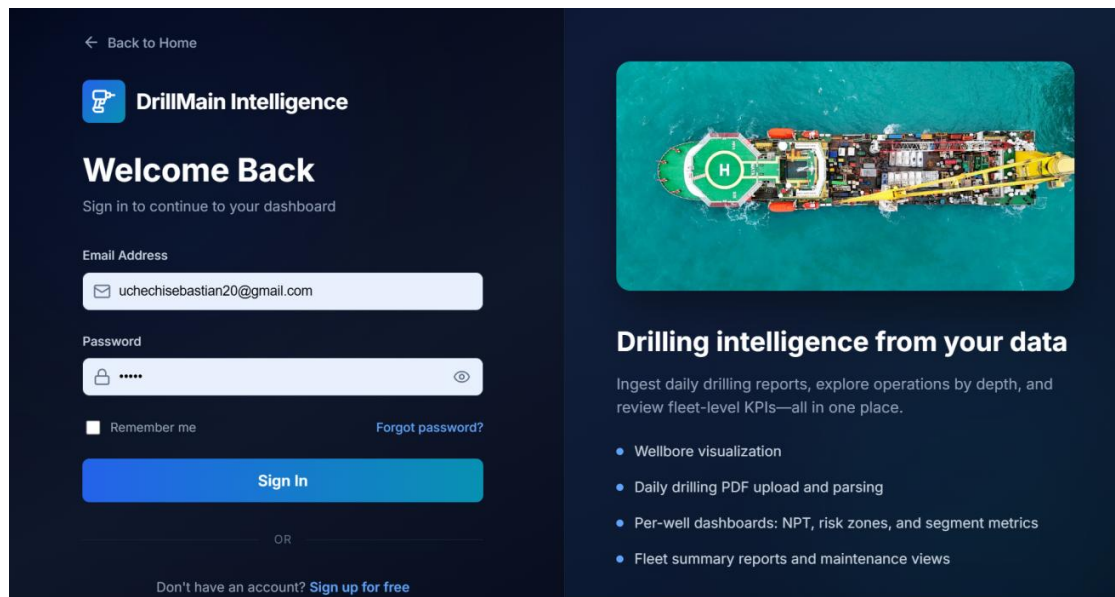
- On the home page, click on the sign-up button to create an account, by inputting your details.



The screenshot shows the 'Create Account' page for DrillMain Intelligence. On the left, there is a hero image of an oil rig with the text 'Join Leading Drilling Teams' and 'Create your free account and start using DrillMain Intelligence today.' Below this, it says 'No credit card required'. On the right, there is a form with the following fields: 'Full name' (pre-filled with 'John Doe'), 'Company' (pre-filled with 'Acme Drilling Co.'), 'Email address' (pre-filled with 'you@company.com'), and 'Password' (with a placeholder 'Create a strong password' and a toggle for visibility). A 'Back to Home' link is at the top left of the form area.

11.7 The login page

- After signing up, you will be directed to the login page.
- Fill in your login details to be able to login.

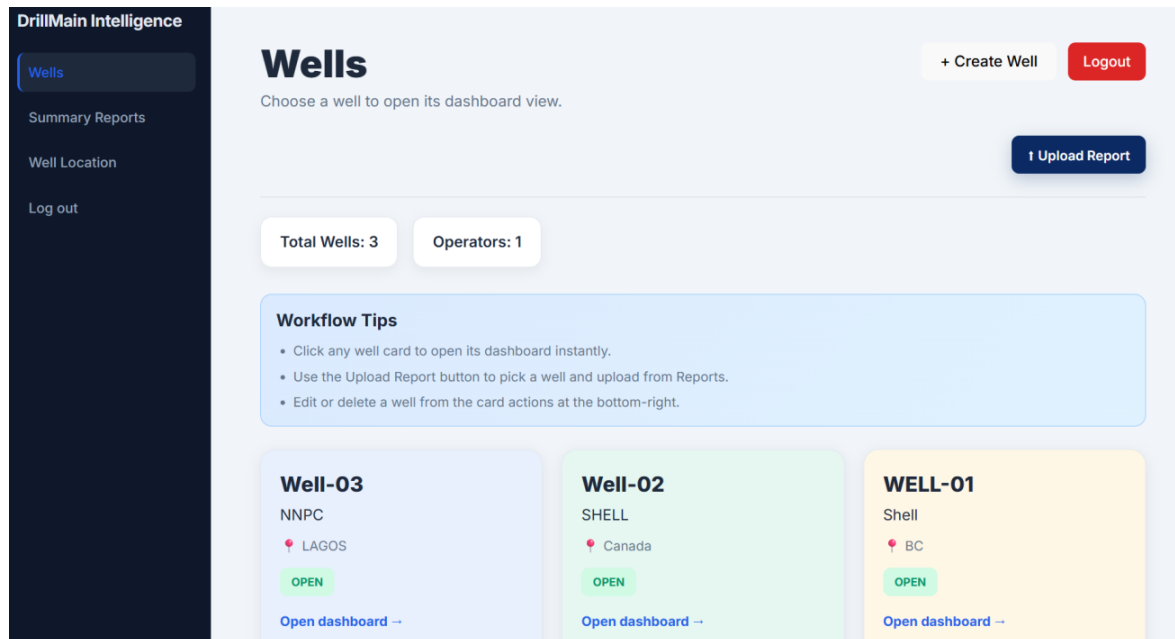


The screenshot shows the 'Welcome Back' login page for DrillMain Intelligence. On the left, there is a login form with the following fields: 'Email Address' (pre-filled with 'uchechisebastian20@gmail.com') and 'Password' (with a placeholder '*****' and a toggle for visibility). Below the password field, there are checkboxes for 'Remember me' and a link for 'Forgot password?'. A blue 'Sign In' button is at the bottom of the form. Below the button, there is an 'OR' separator and a link 'Don't have an account? Sign up for free'. On the right, there is a hero image of an oil rig with the text 'Drilling intelligence from your data' and 'Ingest daily drilling reports, explore operations by depth, and review fleet-level KPIs—all in one place.' Below this, there is a list of features: 'Wellbore visualization', 'Daily drilling PDF upload and parsing', 'Per-well dashboards: NPT, risk zones, and segment metrics', and 'Fleet summary reports and maintenance views'. A 'Back to Home' link is at the top left of the page.

Creation of Wells

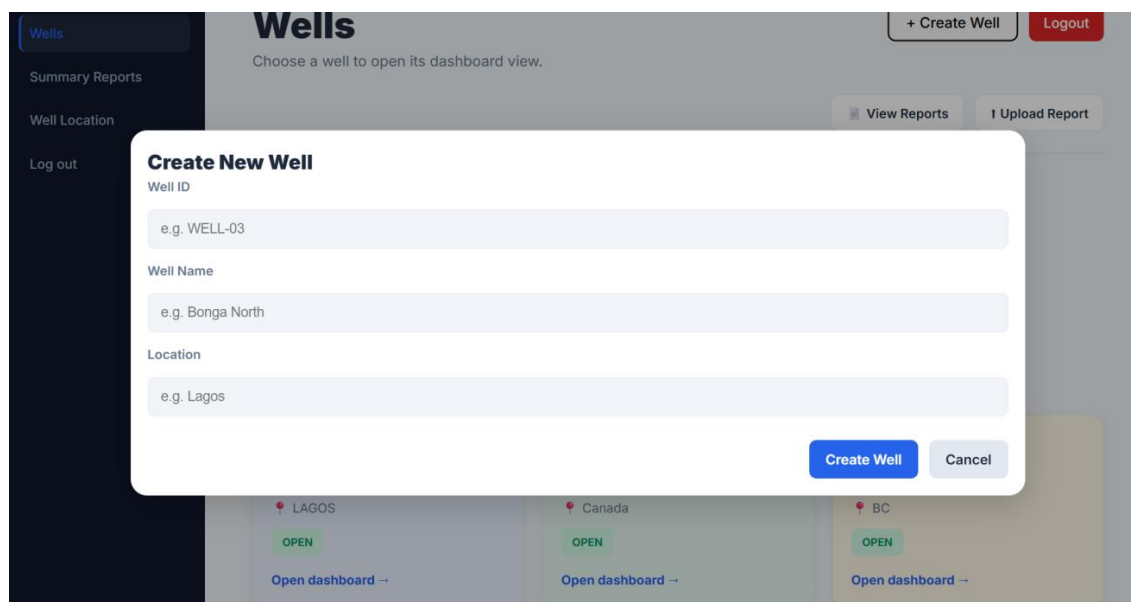
11.8 Wells Page

- After you login, you will be directed to the wells page.



11.9 Create New Well

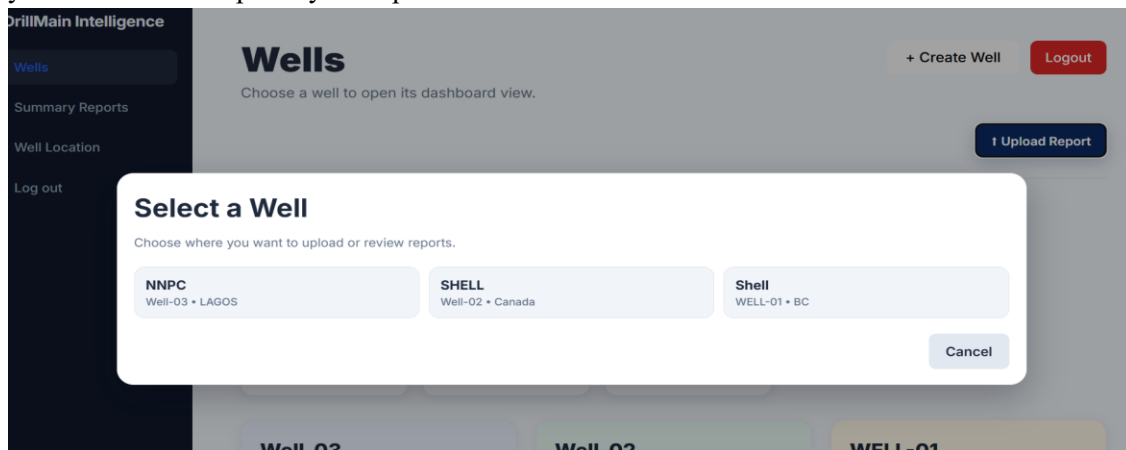
- Click on the create well button to register a new well.
- Fill the well ID, well name and location.
- Click “create well” button to create a new well.



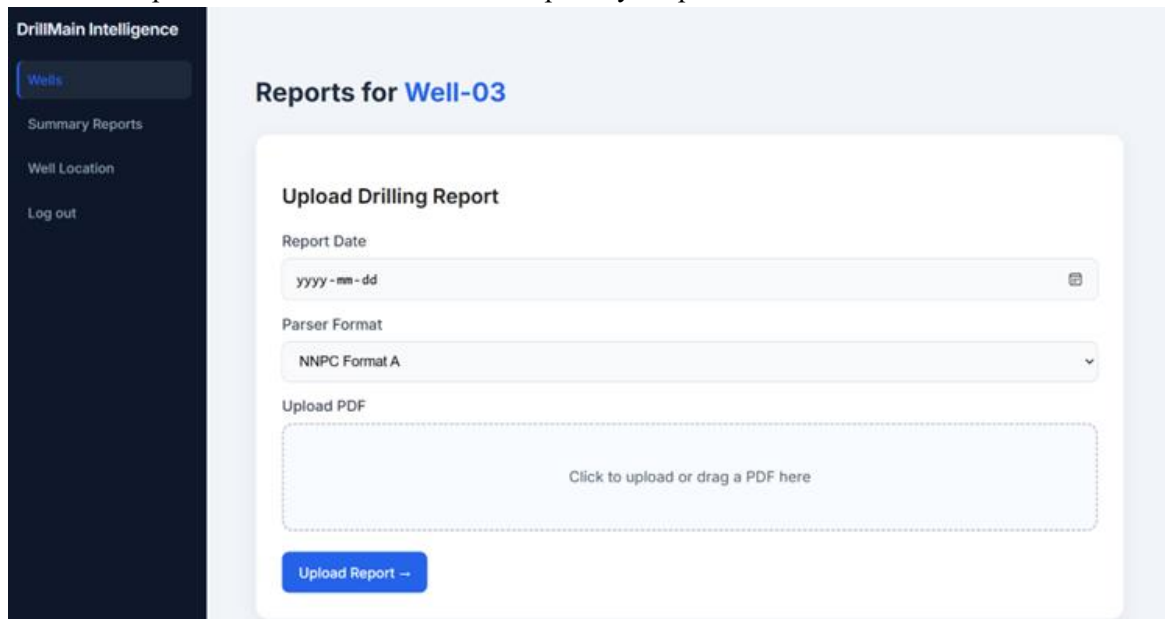
Reports

11.10 uploading Reports

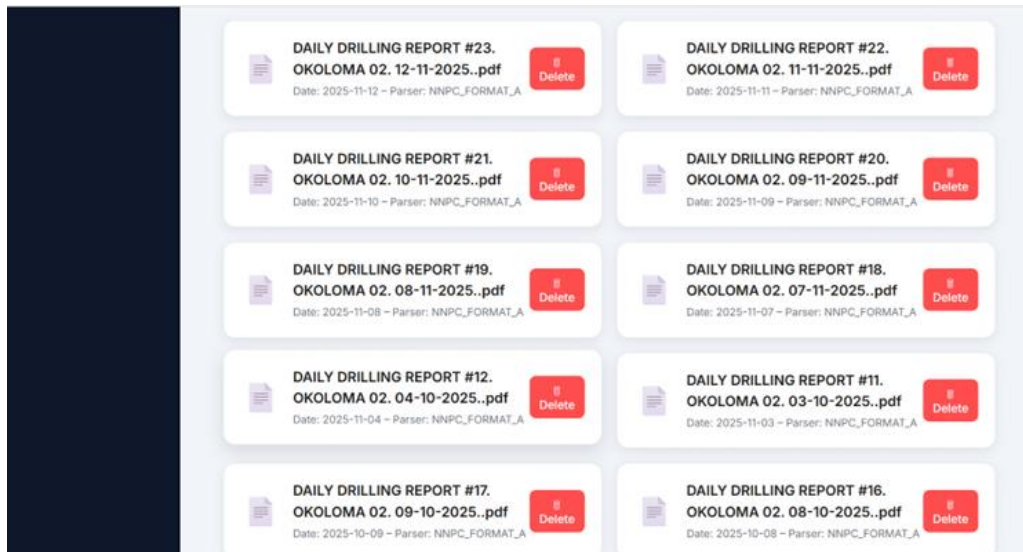
- On the wells page, click on upload report. This will open a window showing the list of wells, click on the well you would like to upload your report.



- After selecting a well, you will be directed to the reports page where you will upload your report.
- Fill in the report date choose the format and upload your pdf.



- Scroll down on the report page to see the uploaded reports.



11.12 Report Details / Editing report

- Click on the report to view the details of the report.

The interface shows the details of Report #20. The sidebar on the left includes the following navigation options:

- DrillMain Intelligence
- Wells
- Summary Reports
- Well Location
- Log out

The main content area displays the following information:

- Report #20 — DAILY DRILLING REPORT #19. OKOLOMA 02. 08-11-2025..pdf**
- Well:** Well-03
- Date:** 2025-11-08
- Parser:** NNPC_FORMAT_A
- Extracted Operations**

Depth From	Depth To	Operation Type	Duration	NPT	Description
6030	6030	Drilling	5.5		PREPARE THE BOP STACK IN SET BACK AREA FOR INSTALLATION. P/U AND LOWER 13-5/8" 5M x 13-5/8" 10M DSA ON 13-3/8" 5M x 13-5/8" 5M CHH AND TIGHTENED CONNECTION BOLTS. P/U & MOVE THE 13-5/8" 10M BOP STACK FROM THE SET BACK AREA & N/U SAME ON 13-5/8" 5M x 13-5/8" 10M DSA AND TIGHTENED CONNECTION BOLTS.
6030	6030	Drilling	17.5		POWERED UP KOOMEY UNIT AND FUNCTION TESTED SAME-OK. R/U DS FRONT LADDER, CATWALK & V-DOOR CONNECTED HYDRAULIC CONTROL LINE FROM KOOMEY UNIT TO BOP. R/U BELL NIPPLE AND SECURE SAME TO THE BOP.

- Click on the “edit report” button to make changes if necessary.
- After clicking on the edit report button, the edit page will open to make changes on the report or delete data.
- Click on “save changes” after making changes to the report.

Well Location
Log out

2025..pdf

Well: Well-03
Date: 2025-11-08
Parser: NNPC_FORMAT_A

Extracted Operations

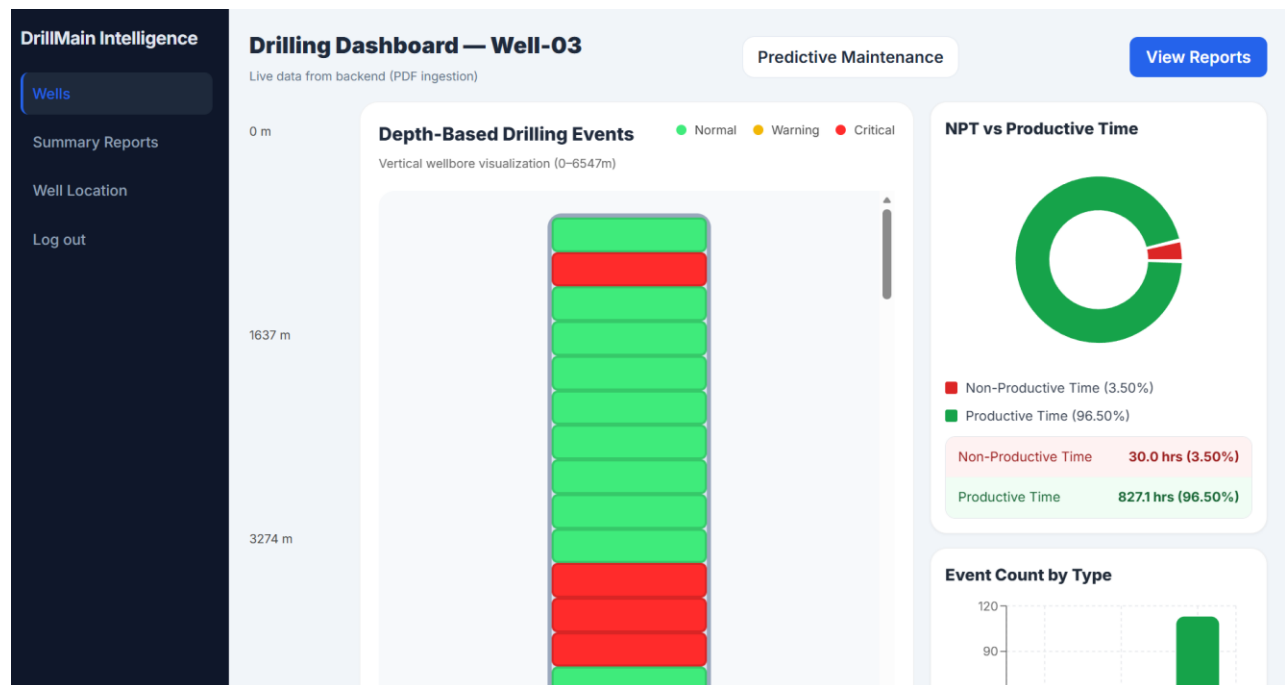
Depth From	Depth To	Operation Type	Duration	NPT	Description	Actions
6030	6030	Drilling	5.5		PREPARE THE BOP STACK IN SET BACK	Delete
6030	6030	Drilling	17.5		POWERED UP KOOKEY UNIT AND FUNCTION	Delete
6030	6030	Drilling	1		FUNCTION ALL FUNCTIONS OF THE BOP	Delete

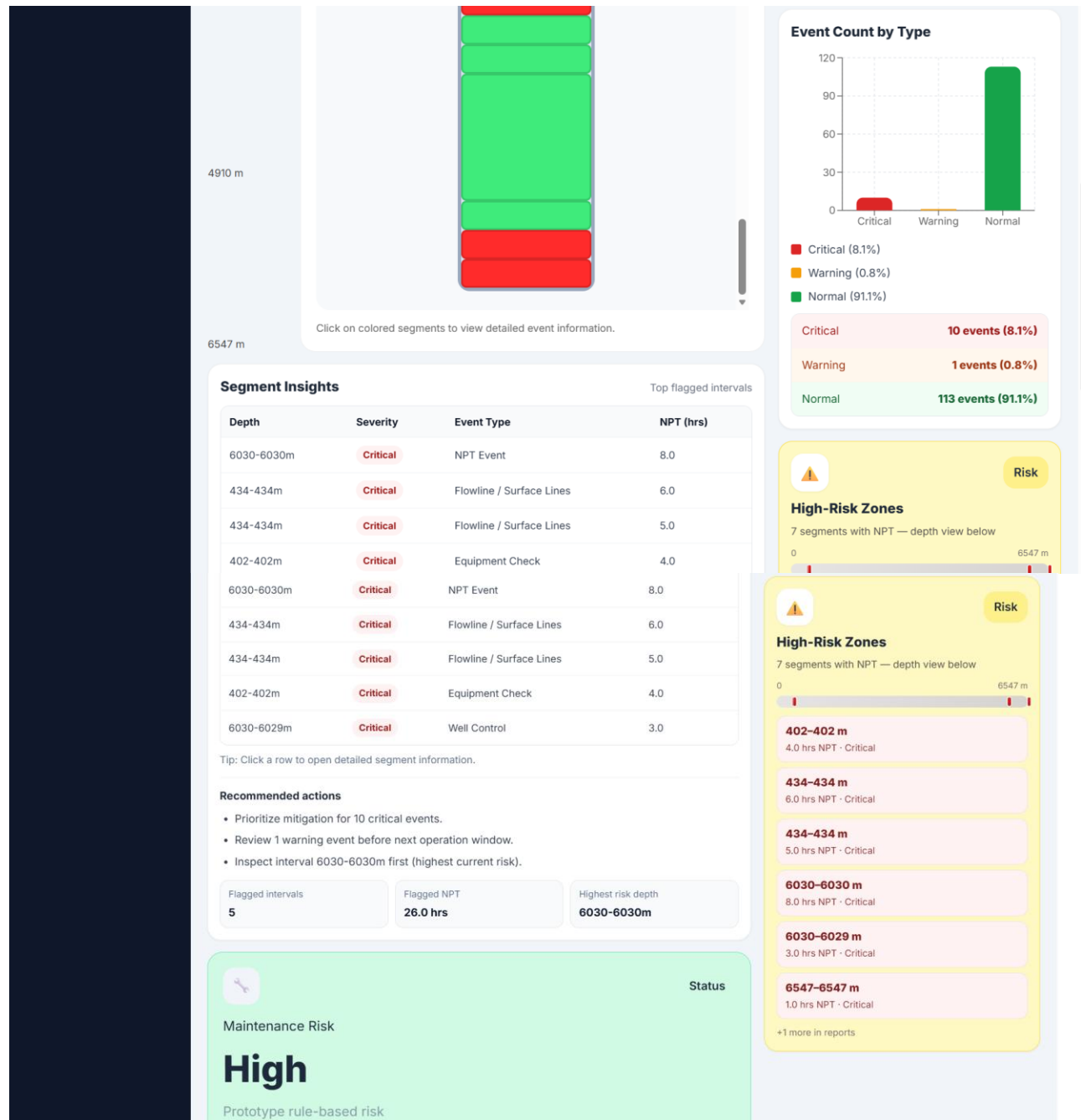
+ Add Row
Save Changes
X Cancel

Dashboard

11.13 Drilling Dashboard

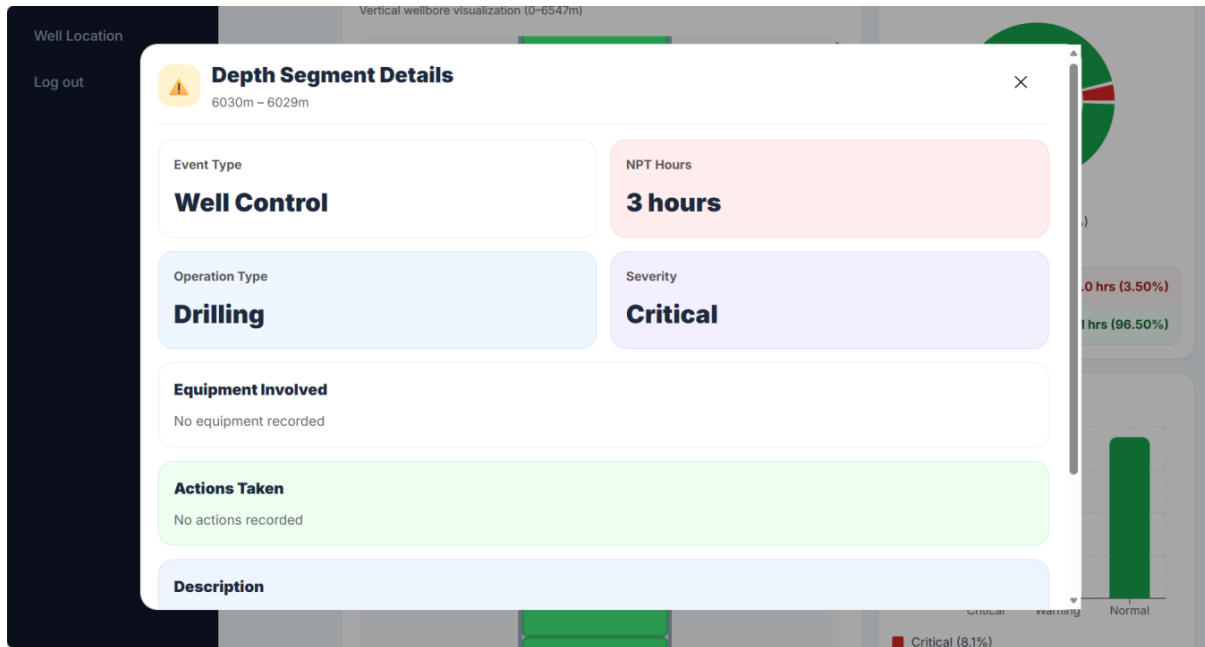
- After uploading and editing the report, go to the wells page and click on the well to view the dashboard.
- The dashboard shows a wellbore drill pipe visualization with segments (the blocks in the vertical pipe), which represents events that occurred in the uploaded report.



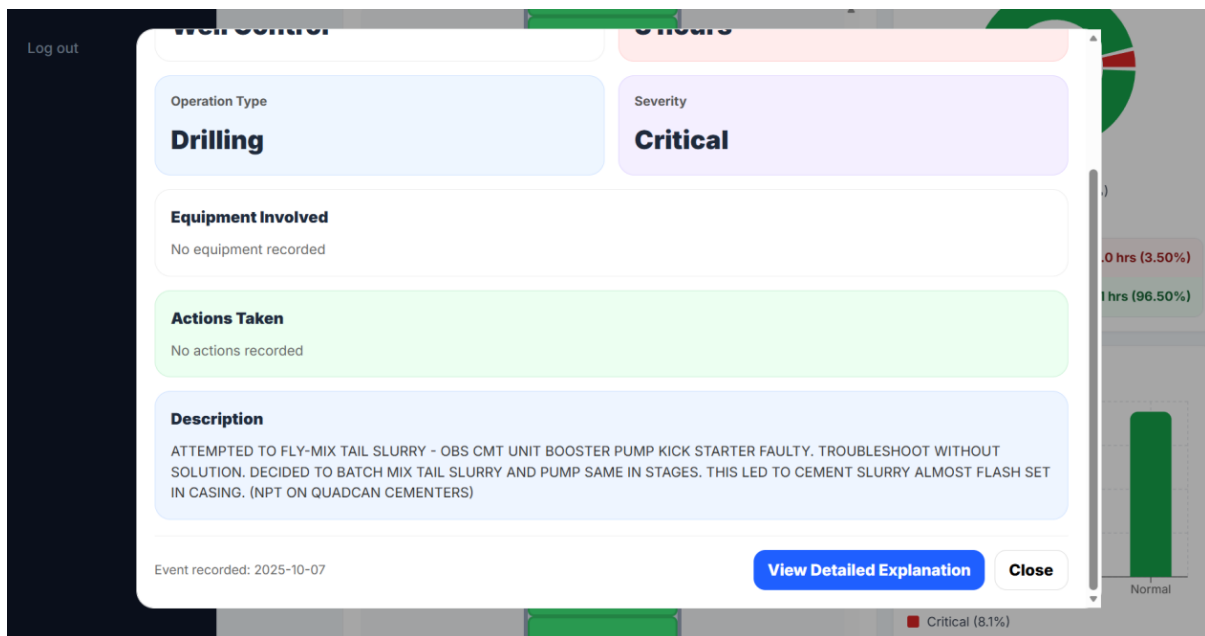


11.14 Segment Details Panel

- Click on the segment and a segment modal will pop up showing a summary of what went on at a particular depth, the severity and the type of event that occurred.



- Click on “View Detailed Explanation” to view the analysis of the event that occurred at a particular depth.



Critical Event Analysis

Depth: 402.0m - 402.0m

How We Determined This Title

The title was set to 'Critical Event Analysis' because the segment severity is critical and no specific event keyword (e.g. stuck pipe) was found in the description.

Why Was This Flagged?

This segment is classified as critical because high non-productive time (4.0 hours; ≥ 2 hours triggers critical classification). The contributing factors below add detail where relevant.

Contributing Factors

High NPT in This Interval

The report explicitly records 4.0 hours of non-productive time for this segment (depth 402.0m–402.0m, operation: 'Drilling'). Our rule: $NPT \geq 2$ hours classifies as critical. Elevated NPT in this depth range indicates operational issues that warrant closer review and mitigation planning.

Similar Events in Well History

Technical Factors Identified

- Formation permeability changes at depth boundary.
- Mud cake buildup in permeable zones.
- Tight clearance between drill string and wellbore.

Recommended Prevention Measures

- ✓ Optimize mud weight for overbalance in this depth range.
- ✓ Minimize static time in this interval.
- ✓ Monitor torque and drag during operations like this one.

Analysis Methodology

This analysis uses the segment's depth interval, operation type, NPT hours, and description from the report. Segment severity (wellbore color) is set by AI when available, otherwise by rule-based logic; the title and flagged explanation match that same level. Contributing factors and recommendations are derived from the report fields.

[Back to Details](#) [Close Analysis](#)

- Click on “Back to details” to go back to the segment modal or click on “Close Analysis” to go back to the dashboard.

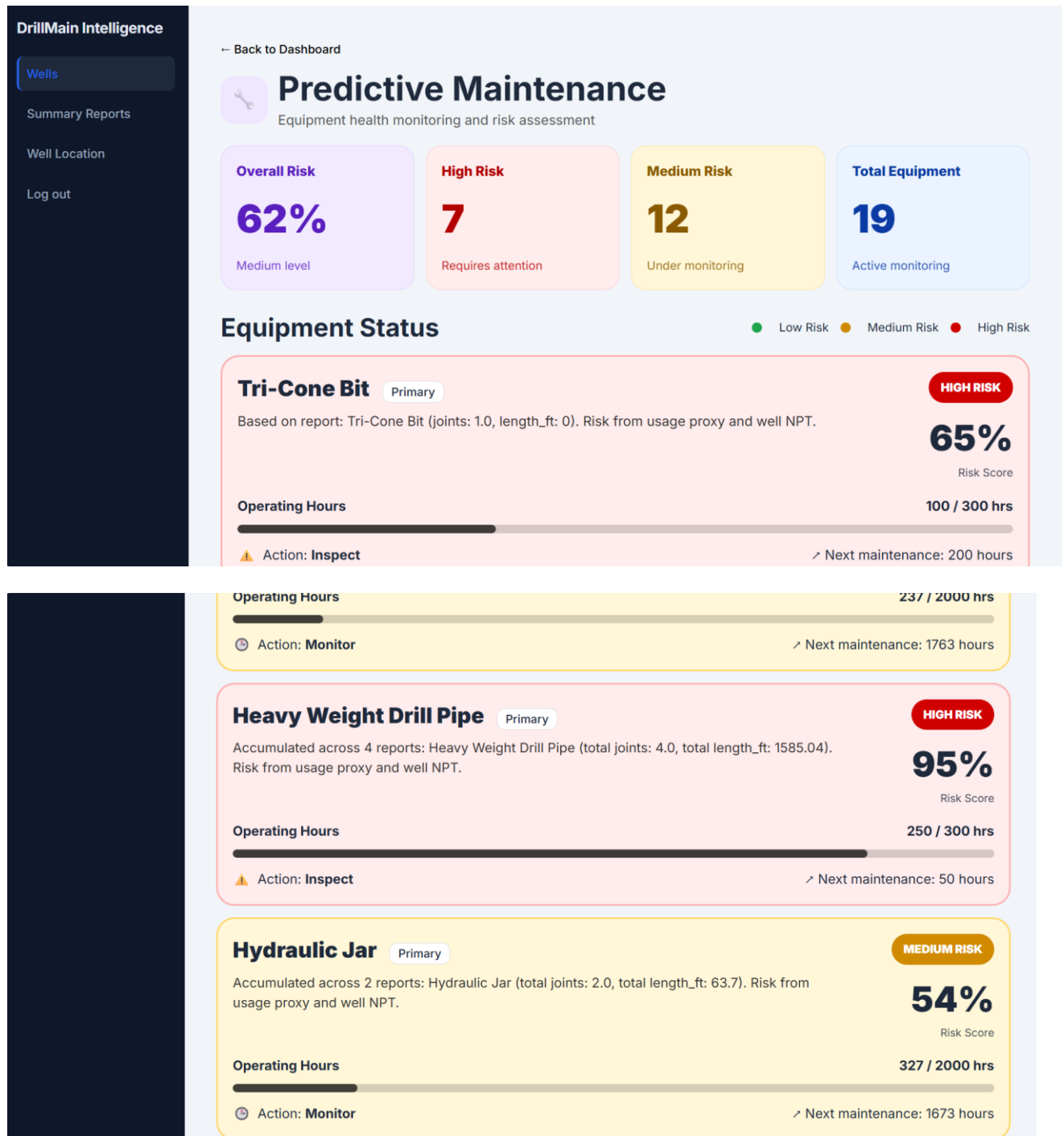
11.15 KPI Cards

- Click on the **NPT vs Productive Time** KPI card on the dashboard to view more information on the Non-productive time.
- Click on the **Event Count by Type** KPI card on the dashboard to view more information on the event count based on the type of event that occurred.

Prédicative Maintenance

11.16 Prédicative Maintenance Page

- Click the predictive maintenance button in dashboard page to open maintenance page for your equipment.



Summary Reports

11.17 Summary Reports page

- Click on “Summary Reports” on the dashboard page in the layout section, to see the summary details of all wells in your account.

Fleet-Wide Reports

Comprehensive analysis across all wells

CSV

PDF

Full Report (JSON)

Active Wells
1

Total NPT
30.0
hrs

Total Events
124

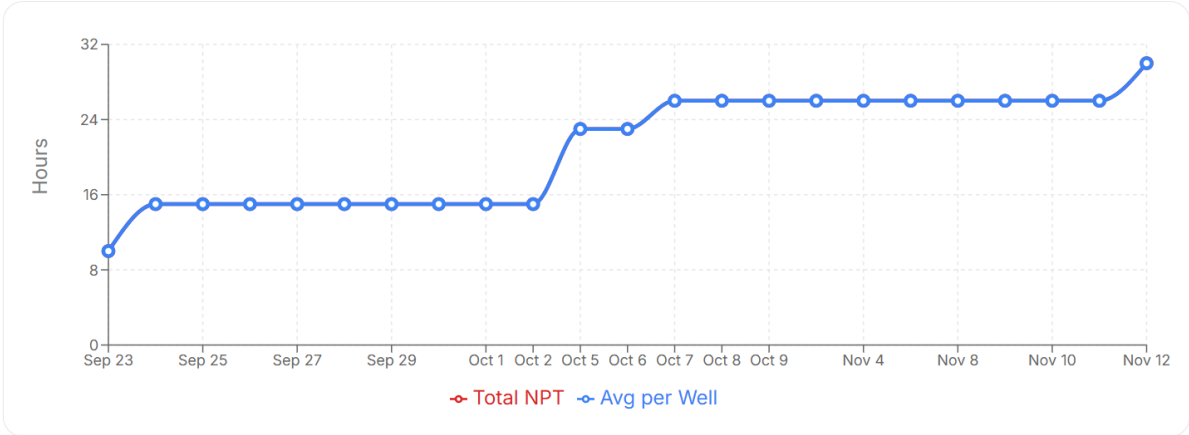
Critical Events
10

Avg Productivity
94.6%

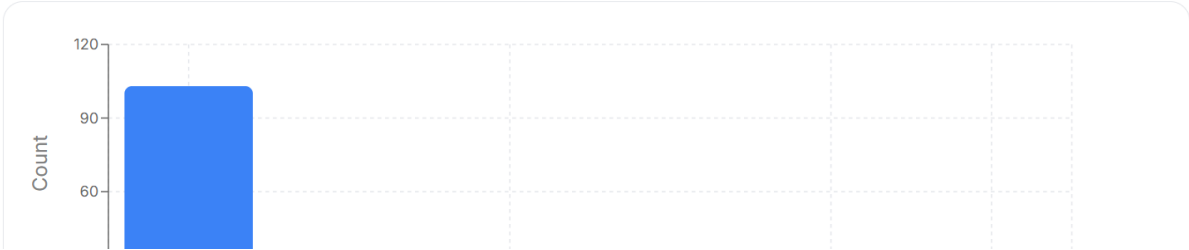
NPT Comparison Across Wells



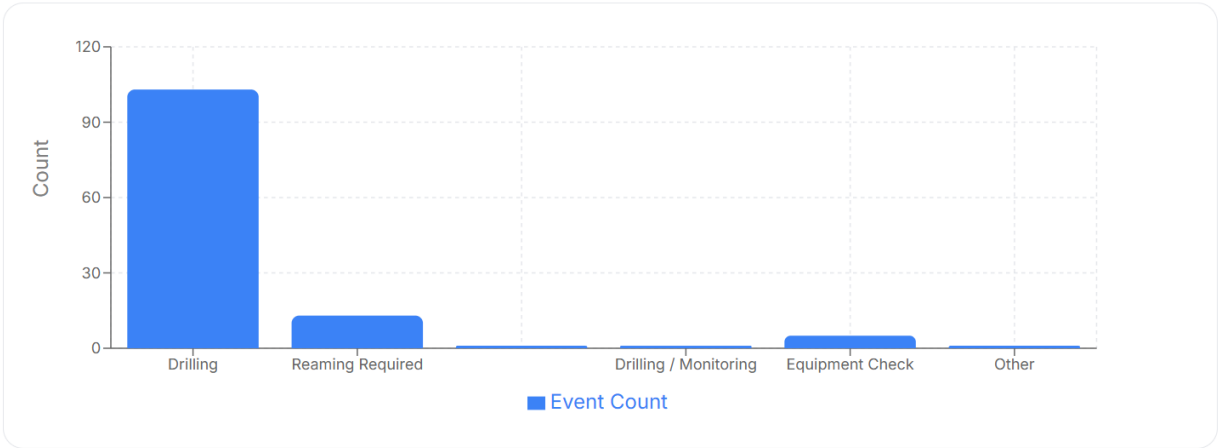
Fleet NPT Trend (Cumulative)



Event Type Distribution (All Wells)



Event Type Distribution (All Wells)



Events per Well



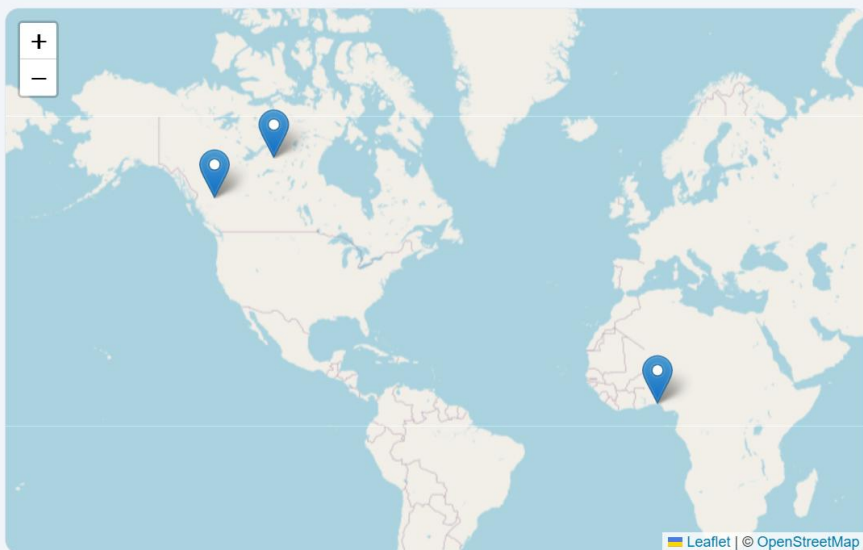
Location

11.18 Well Location page

- Click on “Well Location” on the dashboard page in the layout section, to see the location of all wells in your account on a map.

Well Location

Map overview of well locations. Wells are placed by geocoding their location or name.



Wells on map (3)

NNPC

LAGOS

SHELL

Canada

Shell

BC

Appendix C – AI Prompt(s) Used

- Explain React components, pages, layout, and routing like I'm a beginner, and guide me step-by-step to set up a React app using Vite on Windows.
- I'm building a drilling dashboard UI. Help me create a Week 4 React layout with wells list + navigation. Keep CSS in separate files and explain what each file does.
- I'm getting an import/casing error in Vite/React. Diagnose the error and tell me exactly what files to rename and what import paths to use.
- How do I modify the frontend such that when I click on the view detailed explanation, it goes ahead to show me the detailed explanation and the analytics gotten based on the data provided? Please also explain very well so I can understand how it is done.
- No there is already `<div className="footerBtns"> <button className="primaryBtn" type="button"> View Detailed Explanation </button> <button className="secondaryBtn" type="button" onClick={onClose}> Close </button> </div>` so when you click on the button that's where the three pictures come in.
- Okay, when I put the backend it will generate the analytics on its own without putting any explanations
- I want to put a button to show back to reports after uploading it is successful. And also, the depth is not working again it did not capture the dept correctly. Also, i would like us to put a full designed upload on the report side because the current upload is when you click on the wells and go to view report. but this time i want it directly on the report page under the wells page in the component part.
- so my wellbore pipe segment is meant to show different color codes when it detects that the report that was uploaded has any critical word in it, in the description but it is not showing.

- let us work based on my project. do I need the data to start so that I can do some analytics to connect it to the front end? Or let us start with, obviously you upload your dataset, then the app should receive this dataset and analyze it and show it on the wells page. When you then click on the particular well, it will show you the drilling pipe with different color coding. But first we should work on uploading the dataset first.
- For the uploading of files, these 3 categories, are you uploading the three documents as one csv, that is the; wells, operations, events. Also, most of this dataset have to be prepared after drilling right? that means there has to be some data input into their various tables. Or can the dataset from the company have everything, then the app begins to separate it by itself. That is to say, does it have to be prepared already.
- Good now, here in my wells, it looks so blank apparts from the wells that was created that is showing. What else do you think i can put here in the wells part to make it look more informative rather than showing only wells.
- when you click view report and upload report it shows page not found. can you link it to the exixting ones also move the quick bar to the side or draw a line to differentiate it from the wells. Also i was thinking when you click on view reports you have to select the well first you want before seeing the reports. while upload report when you click on it, it can take us to the one we have already created.
- Okay now i see why the color codes are not showing on the segment. I tried editing one of the decription and i put pack-off inside so that it can display the color code. But i noticed the edit is not updated in the segment but it updated in the report details and backend. so the in the segment if you click the segment modal for each block it shows the original description but doesnt show the one i edited. so my testing did not work. Do you understand what i am trying to explain. so the segment is getting the description dirrectly from the uploaded report not minding if it was edited or not so it doesnt show my update. I edited it cause i just wanted to test the colors on the wellbore. Do you understand? and what can i do now?
- lets continue the upload i think most companies use excel not csv right so

is it better to use excel to upload the data or csv based on what companies use.

- I dont want the exact names because not everyone in the industry would have time to put the exact names.
- okay if I integrate the script into the endpoint upload, that means it is ready for use. what does that mean. Also, after integrating it, can i test it to see if it is working. secondly will the upload be able to carry thousands of roles of data.
- No i want it to show which report or date each NPT came from and put it in a bar chat. in the KPIModal for NPT. For example, the x axis will be the particular reportdate it occurred, why the y axis will be the number of NPT. So when you click on the NPT KpiCard @Dashboards.jsx (95-108) , it goes to the modal and brings out the barchart of each report that has summed up to the total NPT like i have described.
- I think creating the edit button in the report details so i can edit the mud and Equipment table if it did not work correctly. And also be able to save it after editing. I also noticed the mud details did not have the first column - Mud desc. please include it and put the edit table for both.
- The pie chart labels for Normal are getting cut off how can I show the full label outside the slice?
- The legend colors don't match the donut for Warning vs Normal how do I align the legend with the slice colors?
- For the horizontal stacked bar 'Events by Depth Range', how do I change Normal from gray to green and make the legend text match the bar colors?
- How do I add pill-style badges for Type and Severity in the Detailed Event Breakdown table, with red/amber/green like a risk UI
- Right now every account sees the same wells after login how should I scope wells in the API so each user only sees their own?
- We only store the user email in localStorage what's a simple way to tell the backend which user is calling the API?
- I want to add a Summary Reports page that shows an overview of all wells on

one screen so we can see what's going on per well and export it. I'd like to understand how to design this end-to-end: what backend data we need e.g. one summary endpoint vs many calls, how to present it e.g. cards per well, KPIs, and how to add simple charts e.g. pie charts for NPT or risk and a way to download the report for example, print to PDF. Can you walk me through the design and implementation so I can see how frontend and backend fit together?

- On the Summary Reports page, the UI stays on 'Loading summary...' and never finishes. I'd like to understand why that happens: is it usually because the request never completes for example, backend not running or proxy, and how should we handle it in the frontend e.g. timeouts, error messages, and cleaning up when the user leaves the page so the app fails in a clear way instead of hanging?
- When I start the backend I get a SyntaxError on the line with `max(o.depth_to or 0 for o in ops, default=0)`. I'm not sure why the generator expression is invalid here. Can you explain how `max()` works with a generator and a default argument in Python, and what the correct parentheses should be so I can fix it and avoid similar mistakes?
- In my drilling dashboard, the segment modal sometimes says 'no critical or warning keywords were found' even when the segment is red. How can I make the Segment Analysis explanation title, 'Why Was This Flagged?' use the same `segment.level` that drives the wellbore color so the text always matches the color?
- How can I have my backend AI analysis read the segment description and generate a short plain-language explanation of what happened and why it was flagged critical/warning, instead of just repeating the description or using a fixed template?
- If I previously implemented the NPT download as a CSV generator in `KpiModal.jsx`, what's the correct way to refactor it back to a `window.print()`-based PDF workflow so graphs are captured, and where exactly should I put that handler?

- How can i reuse my existing NPT Analytics modal CSS (header, KPI row, section cards, tables) for a new Event Analytics view without duplicating too much styling logic?
- How can i wire the 'Download Event Report' button in my Event Analytics modal so it uses window.print() (like my Summary Reports page) and includes all the event charts and tables in the generated PDF?
- What's the best way to pass wellName, segments, and kpis from my Dashboards.jsx page into a generic KpiModal so that clicking different KPI cards (e.g. NPT vs Event Count) can render completely different analytics views based on the same underlying dashboard data?